



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



Escola Tècnica
Superior d'Enginyeria
Informàtica

Escuela Técnica Superior de Ingeniería Informática
Universidad Politécnica de Valencia

ARQUITECTURA HÍBRIDA RAG-KAG: MEJORA DE SISTEMAS DE RECUPERACIÓN AUMENTADA MEDIANTE GRAFOS DE CONOCIMIENTO

TRABAJO FIN DE MASTER

Master en Inteligencia Artificial y Big Data Analytics

Autor: Andrés Coronado Giménez

Tutor: Lluís Hurtado Oliver; Vicent Ahuir Esteve

Curso 2025/2026

Resum

Els sistemes de generació augmentada mitjançant recuperació (RAG) s'han consolidat com una de les principals eines per ampliar les capacitats dels models de llenguatge mitjançant consultes sobre informació externa. No obstant això, aquests sistemes presenten limitacions quan la informació necessària per respondre una pregunta es troba distribuïda entre múltiples documents o requereix tasques de raonament multisalt. En aquest context, els grafs de coneixement sorgixen com una alternativa capaç de representar explícitament entitats i relacions, facilitant la connexió entre fragments d'informació dispersos.

En aquest treball s'aborda el disseny, desenvolupament i avaluació d'una arquitectura híbrida basada en la recuperació d'informació mitjançant grafs de coneixement per a tasques de resposta a preguntes. L'objectiu principal és estudiar la viabilitat de construir un sistema complet capaç d'extraure coneixement a partir de textos no estructurats, emmagatzemar-lo en una base de dades orientada a grafs i recuperar posteriorment la informació rellevant per respondre preguntes complexes, tot això en un entorn d'execució local i amb recursos computacionals limitats.

Per a això s'utilitza el conjunt de dades 2WikiMultihopQA, àmpliament emprat en tasques de raonament multisalt. A partir dels seus textos es construeix un graf de coneixement utilitzant tècniques d'extracció automàtica d'entitats i relacions mitjançant el model REBEL. La informació extreta s'emmagatzema en Neo4j, on s'apliquen tècniques de normalització, deduplicació d'entitats i recuperació d'informació. El sistema de recuperació proposat es basa en la detecció d'entitats dins de la pregunta, la recuperació de subgrafs de fins a dos salts i un procés de filtratge i reordenació (*reranking*) dissenyat per seleccionar únicament els camins més rellevants per a la resposta.

A més de les mètriques oficials del conjunt de dades, s'empra un sistema d'avaluació addicional orientat a analitzar de manera independent la qualitat de la recuperació d'informació i de la generació de respostes. Per a això es definixen mètriques específiques relacionades amb la cobertura del context recuperat, la detecció d'al·lucinacions i la identificació de respostes evasives.

Els resultats obtinguts mostren la viabilitat de construir i utilitzar sistemes de recuperació basats en grafs de coneixement en entorns locals, així com els avantatges que aporta la representació explícita d'entitats i relacions per a tasques de raonament multisalt. Al mateix temps, el treball posa de manifest que la qualitat de l'extracció automàtica de coneixement constituïx una de les principals limitacions del rendiment global d'aquest tipus d'arquitectures.

A més, els resultats evidencien la necessitat d'avaluar de manera independent la recuperació d'informació i la generació de respostes, ja que les mètriques tradicionals no permeten una mesura precisa del comportament d'aquest tipus d'arquitectures.

Finalment, es proposa una metodologia de desenvolupament per a la construcció, avaluació i anàlisi de sistemes híbrids RAG-KAG que pot servir com a base per a futures investigacions en aquest àmbit.

Paraules clau: RAG, KAG, Graf de coneixement, Recuperació d'informació, LLM, QA

Resumen

Los sistemas de generación aumentada mediante recuperación (RAG) se han consolidado como una de las principales herramientas para ampliar las capacidades de los modelos de lenguaje mediante consultas sobre información externa. Sin embargo, estos sistemas presentan limitaciones cuando la información necesaria para responder a una pregunta se encuentra distribuida entre múltiples documentos o requiere tareas de razonamiento multi-salto. En este contexto, los grafos de conocimiento surgen como una alternativa capaz de representar explícitamente entidades y relaciones, facilitando la conexión entre información dispersa.

En este trabajo se aborda el diseño, desarrollo y evaluación de una arquitectura híbrida basada en recuperación de información mediante grafos de conocimiento para tareas de pregunta-respuesta. El objetivo principal es estudiar la viabilidad de construir un sistema completo capaz de extraer conocimiento de textos no estructurados, almacenarlo en una base de datos orientada a grafos y recuperar posteriormente la información relevante para responder preguntas complejas, en un entorno de ejecución local y con recursos computacionales limitados.

Para ello se emplea el dataset 2WikiMultihopQA, ampliamente utilizado en tareas de razonamiento multi-salto. A partir de sus textos se construye un grafo de conocimiento utilizando técnicas de extracción automática de entidades y relaciones mediante el modelo REBEL. La información extraída se almacena en Neo4j, donde se aplican técnicas de normalización, deduplicación de entidades y recuperación de información. El sistema de *retrieval* propuesto se basa en la detección de entidades dentro de la pregunta, la recuperación de subgrafos de hasta dos saltos y un proceso de filtrado y *reranking* diseñado para seleccionar únicamente los caminos más relevantes para la respuesta.

Además de las métricas oficiales del dataset, se emplea un sistema de evaluación adicional enfocado en analizar de forma independiente la calidad de la recuperación de información y de la generación de respuestas. Para ello, se definen métricas específicas relacionadas con la cobertura del contexto recuperado, la detección de alucinaciones y la identificación de respuestas evasivas.

Los resultados obtenidos muestran la viabilidad de construir y utilizar sistemas de recuperación basados en grafos de conocimiento en entornos locales, así como las ventajas que aporta la representación explícita de entidades y relaciones para tareas de razonamiento multi-salto. Al mismo tiempo, el trabajo expone cómo la calidad de la extracción automática de conocimiento constituye una de las principales limitaciones del rendimiento global de este tipo de arquitecturas.

Además, los resultados exponen la necesidad de evaluar de forma independiente la recuperación de información y la generación de respuestas, ya que las

métricas tradicionales no permiten una medición precisa del comportamiento de este tipo de arquitecturas.

Finalmente, se propone una metodología de desarrollo para la construcción, evaluación y análisis de sistemas híbridos RAG-KAG que puede servir como base para futuras investigaciones en este ámbito.

Palabras clave: RAG, KAG, Grafo de Conocimiento, Recuperación de información, LLM, QA

Abstract

Retrieval-Augmented Generation (RAG) systems have become one of the main approaches for extending the capabilities of large language models by enabling access to external information. However, these systems present important limitations when the information required to answer a question is distributed across multiple documents or requires multi-hop reasoning. In this context, knowledge graphs emerge as an alternative capable of explicitly representing entities and their relationships, facilitating the connection of scattered pieces of information.

This work addresses the design, development, and evaluation of a hybrid architecture based on knowledge graph information retrieval for question-answering tasks. The main objective is to study the feasibility of building a complete system capable of extracting knowledge from unstructured text, storing it in a graph database, and subsequently retrieving the relevant information needed to answer complex questions, all within a local execution environment with limited computational resources.

To this end, the 2WikiMultihopQA dataset, widely used for multi-hop reasoning tasks, is employed. A knowledge graph is constructed from its documents using automatic entity and relation extraction techniques based on the REBEL model. The extracted information is stored in Neo4j, where normalization, entity deduplication, and information retrieval techniques are applied. The proposed retrieval system is based on entity detection within the input question, retrieval of subgraphs up to two hops away, and a filtering and reranking process designed to select only the most relevant paths for answering the question.

In addition to the official dataset metrics, an additional evaluation framework is proposed to independently assess the quality of both information retrieval and answer generation. To achieve this, specific metrics are defined to evaluate retrieved context coverage, hallucination detection, and the identification of evasive responses.

The obtained results demonstrate the feasibility of building and deploying knowledge graph-based retrieval systems in local environments, as well as the advantages provided by the explicit representation of entities and relationships for multi-hop reasoning tasks. At the same time, the study shows that the quality of automatic knowledge extraction constitutes one of the main limitations affecting the overall performance of this type of architecture.

Furthermore, the results highlight the importance of independently evaluating information retrieval and answer generation, since traditional evaluation metrics do not provide an accurate assessment of the behavior of this type of architecture.

Finally, this work proposes a development methodology for the construction, evaluation, and analysis of hybrid RAG-KAG systems that may serve as a foundation for future research in this field.

Key words: RAG, KAG, Knowledge Graph, Information Retrieval, LLM, QA

Índice general

Índice general	VII
Índice de figuras	IX
Índice de tablas	IX

1	Introducción	1
1.1	Objetivos	2
1.2	Impacto esperado	3
1.3	Metodología	3
1.4	Estructura de la memoria	4
2	Estado del Arte	7
3	Arquitectura propuesta	11
3.1	Descripción general	11
3.2	Selección del dataset	12
3.3	Extracción de conocimiento	15
3.3.1	Retos de la extracción	15
3.3.2	Herramientas de extracción	16
3.4	Construcción del grafo	18
3.4.1	Modelo de datos	19
3.4.2	Carga de los datos	20
3.4.3	Grafos de referencia	20
3.4.4	Análisis comparativo entre grafos	21
3.4.5	Normalización y deduplicación	22
3.5	Diseño del retrieval	23
3.5.1	Consultas al grafo	23
3.5.2	Recuperación basada en entidades	24
3.5.3	Deduplicación	26
3.5.4	Recuperación y filtrado de información	28
3.5.5	Generación de la respuesta	33
3.6	Métricas de evaluación	34
3.6.1	Limitaciones de las métricas oficiales	34
3.6.2	Métricas adicionales	35
3.6.3	Definición de las métricas	35
3.6.4	Evaluador externo	37
4	Resultados	41
4.1	Hardware utilizado y limitaciones	41
4.2	Resultados grafo de referencia	42
4.3	Resultados grafo generado con REBEL	44
4.3.1	Resultados globales	44
4.3.2	Resultados por tipo de pregunta	49

4.3.3	Análisis de rendimiento y recursos	51
5	Conclusiones	55
5.1	Limitaciones	56
5.2	Trabajo futuro	57
	Bibliografía	59

Índice de figuras

3.1	Flujo arquitectura	12
3.2	Proceso de construcción del grafo de conocimiento	15
3.3	Proceso de recuperación de la información	32

Índice de tablas

3.1	Comparativa entre los grafos de referencia y el construido con REBEL	21
3.2	Componentes del sistema evaluados por cada una de las métricas utilizadas.	37
3.3	Resumen de las métricas utilizadas y su definición.	37
3.4	Tasa de aciertos de los modelos como LLM-as-judge	39
4.1	Resultados métricas dataset grafo de referencia	43
4.2	Resultados métricas propias grafos de referencia	43
4.3	Comparativa grafos de referencia con grafo real	45
4.4	Resultados obtenidos mediante las métricas oficiales de 2WikiMultihopQA	46
4.5	Resultados obtenidos mediante las métricas adicionales propuestas	47
4.6	Métricas derivadas para el análisis del comportamiento del sistema	48
4.7	F1 por tipo de pregunta	49
4.8	Context Recall por tipo de pregunta	50
4.9	Correctness Supported por tipo de pregunta	50
4.10	Missed Retrieval por tipo de pregunta	51
4.11	Estadísticas de los tiempos de recuperación por pregunta	52
4.12	Comparativa de consumos RAM	53

CAPÍTULO 1

Introducción

Los modelos de lenguaje de gran tamaño (LLMs) han supuesto un gran avance en las tareas de generación de texto y en sistemas de pregunta-respuesta, pasando a ser una herramienta imprescindible para tareas tanto de uso personal como académico y empresarial. Sin embargo, su conocimiento se ve limitado por los datos disponibles durante el entrenamiento, por lo que su adaptación en entornos específicos sigue siendo un desafío. Inicialmente, las técnicas de *fine-tuning* mitigaban este problema, permitiendo aumentar el conocimiento disponible de los modelos y adaptarlos a tareas concretas. Estas técnicas, aunque efectivas, son costosas computacionalmente y no tienen una adaptación dinámica en entornos de constante cambio. Debido a estas limitaciones surgen los sistemas basados en la generación aumentada por recuperación (RAG), donde la información adicional se consulta en el momento de la generación, sin necesidad de entrenamiento, pudiendo así tener acceso a información actualizada en cada momento. Es por esto que ha sido ampliamente aceptado en entornos donde la información externa no es estática sino que se actualiza a lo largo del tiempo.

A pesar de las ventajas introducidas, los sistemas RAG no son una solución definitiva a estos problemas y presentan limitaciones importantes. En estos sistemas la búsqueda de la información relevante para la tarea se realiza por similitud semántica, lo que los hace muy efectivos para respuestas donde el contexto se encuentra explícitamente en la pregunta, pero débiles en escenarios donde se requiere un razonamiento complejo, integración de diversas fuentes o inferencias multi-salto en los que la información solicitada se encuentra dispersa en diferentes documentos o fuentes.

Con el objetivo de mitigar estas limitaciones aparecen las soluciones basadas en grafos de conocimiento (KAG). En estos grafos la información se modela con una estructura de entidades y relaciones, lo que permite tener una conexión explícita entre toda la información de una o varias fuentes de datos. Esta estructura, en combinación con los sistemas de RAG clásicos, permite encontrar relaciones que de otra forma quedan ocultas, lo que permite a los modelos de lenguaje realizar inferencias complejas, siendo más adecuado para tareas que requieren razonamiento avanzado.

Por otro lado, el uso de estas herramientas requiere en muchos casos el uso de soluciones en línea, con el uso de modelos en la nube. Esto implica serios problemas en relación con la privacidad, donde los riesgos de proveer informa-

ción sensible a terceros son inaceptables. Además, también se incurre en costes económicos muchas veces inasumibles. Es por esto, que el desarrollo y despliegue de arquitecturas en entornos locales se presenta como un requisito imprescindible para tener un control sobre información sensible y reducir la dependencia de servicios externos.

En este contexto, este trabajo está motivado por el desarrollo y despliegue de sistemas híbridos de RAG apoyados en grafos de conocimiento implementados en entornos locales. Esto implica requerimientos técnicos más estrictos donde los recursos son limitados y los requisitos de privacidad elevados.

1.1 Objetivos

Los objetivos principales de este trabajo son, por un lado, el diseño, despliegue y evaluación de una arquitectura híbrida de recuperación de información, y por otro, el estudio de la viabilidad de implementación en un entorno local con recursos limitados. Este sistema debe poder ser implementado y utilizado sin el recurrir a herramientas en la nube y con el uso de recursos limitados. La principal aportación del sistema consiste en el desarrollo de una base de datos orientada a grafos a partir de textos de diversas áreas sin un esquema predefinido y la posterior recuperación de información.

Para la consecución de esta tarea se plantean varios objetivos específicos entre los que se encuentran:

- Diseñar un proceso de extracción automática a partir de textos en lenguaje natural. Este diseño debe identificar entidades relevantes del texto, así como sus relaciones, sin depender de un esquema fijo.
- Integrar esta información dentro de una base de datos que permita la consulta y extracción de la información relevante para la tarea en cuestión.
- Desarrollar el sistema de recuperación, donde se deben encontrar y filtrar las entidades y relaciones relevantes.
- Preparar la información recuperada para su utilización por un modelo de lenguaje y así responder a las preguntas planteadas por el usuario.
- Integrar todos los componentes dentro de una arquitectura que permita la extracción de la información, el almacenamiento y la posterior recuperación.
- Definir e implementar métricas que permitan evaluar tanto la calidad del proceso de extracción de conocimiento como la calidad de la recuperación de esta información, así como la calidad de las respuestas generadas.
- Analizar las posibilidades de implementación en función de los recursos disponibles. Para ello se debe evaluar el consumo de los diferentes componentes y localizar los cuellos de botella del sistema hasta encontrar un equilibrio entre rendimiento y consumo de recursos.

1.2 Impacto esperado

Con este trabajo se espera contribuir al estudio del proceso completo de integración de grafos de conocimiento en sistemas de recuperación de información basados en modelos de lenguaje. Se pretende proporcionar una metodología para el desarrollo y evaluación de estos sistemas exponiendo las ventajas y limitaciones frente a sistemas de recuperación de la información basados en búsquedas semánticas.

Desde el punto de vista técnico se espera poder evaluar el comportamiento la calidad de los sistemas de pregunta-respuesta, especialmente en los casos donde se requiere combinar información dispersa como en el caso de preguntas multi-salto. Con este estudio se pretende también identificar los desafíos y limitaciones de estos sistemas, especialmente en la construcción automática de grafos a partir de información no estructurada.

Por otro lado, se quiere demostrar la posibilidad de implementar un sistema completamente local, estableciendo los límites en cuanto a recursos utilizados. Esto puede servir como base para futuros estudios con requerimientos similares o para cualquier persona que desee poder contar con un sistema propio en cualquier ordenador personal.

Por último, se pretende establecer, mediante el desarrollo de una arquitectura completa, una metodología reproducible para futuras investigaciones en este área, permitiendo partir de una base ya desarrollada y testeada.

1.3 Metodología

La metodología de este trabajo se basa en desarrollar los distintos componentes del sistema para obtener una arquitectura completa. Esto abarca todo el proceso, desde una fase inicial de selección de los datos, continuando con el posterior desarrollo de la lógica del sistema y finalizando con la evaluación y comparación de resultados.

La primera etapa se centra en la búsqueda de un conjunto de datos apropiado para la tarea de construcción de un grafo de conocimiento. Los datos escogidos deben contener suficiente información para poder construir un grafo poblado y diverso. También deben disponer de una base de referencia para poder partir de una situación ideal y poder comparar los resultados con ella. Esta base debe permitir evaluar tanto la estructura del grafo como las respuestas esperadas a las cuestiones planteadas.

Tras seleccionar el conjunto de datos comienza la fase de extracción de conocimiento y construcción del grafo. Se estudiarán diferentes métodos de extracción automática de tripletas a partir de textos no estructurados de diversas áreas, para posteriormente integrarlas en una base de datos orientada a grafos.

Una vez construido el grafo de conocimiento comienza el diseño del sistema de recuperación. Se estudiarán técnicas para la búsqueda de información relevante en el grafo a partir de preguntas en lenguaje natural para su posterior filtrado,

selección y estructuración, con el objetivo de eliminar la información irrelevante que pueda deteriorar la respuesta final.

Posteriormente, con la información recuperada, se pasa a la fase de generación de la respuesta, para lo que se hará uso de un modelo de lenguaje. Aquí se analizarán distintas formas de pasar la información al modelo, así como el rendimiento de distintos modelos ejecutados localmente.

Con las distintas partes del sistema desarrolladas se implementarán diversas métricas que permitan la evaluación de los diferentes componentes del sistema. Esto incluye la calidad del proceso de extracción de información sobre el grafo, así como las respuestas generadas. La selección de métricas deberá poder evaluar independientemente el rendimiento de los distintos elementos del sistema.

Por último se realizará una comparación de los resultados obtenidos en función de los métodos explorados con anterioridad y de las distintas herramientas utilizadas. Este análisis permitirá valorar las ventajas y limitaciones de cada alternativa y determinar que combinaciones ofrecen mejores resultados en términos de calidad, rendimiento y consumo de recursos.

1.4 Estructura de la memoria

El desarrollo principal del trabajo se recoge en el Capítulo 3, dedicado al diseño e implementación de la arquitectura. En él se describe de forma detallada el proceso completo seguido para construir el sistema y su posterior evaluación.

En primer lugar, en la Sección 3.2 se justifica la selección del conjunto de datos utilizado y sus fortalezas a la hora de evaluar tareas de recuperación de información y razonamiento multi-salto. A continuación, en la Sección 3.3, se aborda el proceso de extracción automática de conocimiento a partir de texto no estructurado, analizando las dificultades propias de esta tarea y las distintas alternativas exploradas hasta la selección de la solución final.

Más adelante, en la Sección 3.4 se describe la construcción de la base de datos orientada a grafos, incluyendo el diseño para la representación de entidades y relaciones, así como las técnicas empleadas para la normalización y deduplicación de la información extraída. Una vez construido el grafo de conocimiento, en la Sección 3.5, se explica el sistema de recuperación de información desarrollado, en la que se describen los mecanismos de detección de entidades, las técnicas de búsqueda empleadas dentro del grafo, la recuperación de subgrafos, los procesos de filtrado y *reranking* de la información y la preparación del contexto final utilizado para responder a las preguntas planteadas.

Por último, en la Sección 3.6 se describen tanto las métricas oficiales utilizadas por el dataset como las métricas adicionales diseñadas para evaluar de forma independiente la calidad del *retrieval* y de la generación de respuestas, así como las herramientas utilizadas.

A continuación se presenta el Capítulo 4, enfocado en los resultados, donde se exponen y analizan los experimentos realizados sobre los distintos componentes del sistema. Se comparan diferentes configuraciones, modelos y estrategias estudiadas durante el desarrollo, evaluando su impacto sobre la calidad de la

extracción de conocimiento, la recuperación de información y la generación de respuestas. Además, se estudia la viabilidad de ejecutar este tipo de arquitecturas en entornos locales con recursos limitados.

Finalmente, la memoria concluye con un capítulo dedicado a las conclusiones y líneas de trabajo futuro, donde se resumen las principales aportaciones realizadas, las limitaciones encontradas y las posibles mejoras que podrían incorporarse en futuras investigaciones.

CAPÍTULO 2

Estado del Arte

El desarrollo de sistemas de recuperación de información integrados con los LLMs es un campo de gran interés dentro del procesamiento del lenguaje natural, que ha experimentado un gran crecimiento en los últimos años. El trabajo Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [1] fue uno de los puntos de partida donde se propuso una arquitectura que combinaba los modelos generativos con información recuperada en el momento de la generación a partir de documentos externos al entrenamiento. Esto permitía reducir la dependencia de los modelos de lenguaje respecto a los datos utilizados durante su entrenamiento y fue el inicio del desarrollo de los sistemas RAG.

A partir de aquí surgieron diversas técnicas dedicadas a mejorar la recuperación de la información externa al modelo. Los sistemas de recuperación basados en búsqueda semántica a través de embeddings fueron una de las técnicas más extendidas, destacando el trabajo Dense Passage Retrieval for Open-Domain Question Answering [2], donde se demostró la mejora significativa en tareas de pregunta-respuesta de dominio abierto. Pero, aunque estos sistemas mejoraron la recuperación de documentos, seguían dependiendo de representaciones vectoriales incapaces de abordar tareas que implican conocer relaciones complejas entre entidades.

Al mismo tiempo, los grafos de conocimiento han sido ampliamente utilizados para la representación estructurada de información. Recursos como DBpedia o Wikidata demostraron la utilidad de tratar la información mediante entidades y relaciones explícitas para tareas de búsqueda, recomendación y razonamiento. Tradicionalmente, la construcción de los grafos de conocimiento se realizaba mediante sistemas independientes de reconocimiento de entidades y extracción de relaciones. Sin embargo, trabajos más recientes como REBEL: Relation Extraction By End-to-end Language Generation [11] propusieron un nuevo enfoque capaz de extraer al mismo tiempo entidades y relaciones utilizando modelos transformadores, lo que permitía la extracción automática de conocimiento a partir de documentos.

La unión de los modelos de lenguaje, ya ampliamente extendidos, con el uso de grafos de conocimiento marcó el inicio de un nuevo área centrada en la recuperación de información a partir de grafos conocida como Knowledge-Augmented Generation (KAG), ampliando las capacidades de los sistemas RAG diseñados hasta ese momento. Uno de los primeros artículos sobre el tema es From

Local to Global: A GraphRAG Approach to Query-Focused Summarization [12], desarrollado por Microsoft Research. En él se introduce la construcción automática de grafos de conocimiento a partir de documentos utilizando modelos de lenguaje y se analizan las limitaciones de los sistemas RAG utilizados hasta el momento. A partir de aquí surge la tecnología de Microsoft GraphRAG, arquitectura que incorpora un grafo de conocimiento construido automáticamente para modelar información contenida en los documentos y mejorar el proceso de recuperación.

A partir de aquí se desarrollan nuevos trabajos en este área, mejorando y ampliando las capacidades de los sistemas RAG-KAG basados en grafos. Entre ellos destacan dos artículos donde se describen de forma general las partes y funcionamiento de este tipo de arquitecturas. En el artículo Retrieval-Augmented Generation with Knowledge Graphs: A Survey [4] se ofrece una visión general de los sistemas RAG basados en grafos, donde se describen sus principales componentes, algunas formas de evaluación típicas y los retos de este campo. Por otro lado, en Graph Retrieval-Augmented Generation: A Survey [5] se revisan las principales arquitecturas GraphRAG y se realiza una clasificación de los grafos en función de su construcción, recuperación de información e integración en los modelos de lenguaje.

El presente trabajo se apoya en algunos de estos estudios, como Knowledge Graph-Guided Retrieval Augmented Generation [3] donde se propone una extensión de los sistemas RAG mediante grafos de conocimiento. Para ello se realiza una expansión del contexto a partir de los fragmentos inicialmente recuperados por búsqueda semántica, contexto que después se organiza para introducirlo al LLM, filosofía similar a la empleada en este trabajo.

En esta línea también aparecen propuestas como HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction [6], que combina recuperación vectorial y basada en grafos de conocimiento. De esta forma, es posible aprovechar al mismo tiempo la similitud semántica y las relaciones estructuradas. Otro trabajo relacionado es G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering [7], orientado a tareas de pregunta-respuesta sobre grafos textuales mediante la recuperación de los subgrafos más relevantes antes de la generación de la respuesta.

Al mismo tiempo, la compleja evaluación de sistemas RAG y GraphRAG pasa a convertirse en un campo de investigación independiente. La necesidad de evaluar tanto la construcción del grafo como la fase de recuperación y generación de la respuesta motivan varios estudios en este ámbito, donde se critican algunos de los benchmarks tradicionales y se proponen nuevas soluciones. Entre estos destaca algunos como Evaluating Knowledge Graph Based Retrieval Augmented Generation Methods under Knowledge Incompleteness [8], donde se estudia el impacto de la falta de datos de los grafos de conocimiento sobre el rendimiento de los sistemas KG-RAG; How Significant Are the Real Performance Gains? An Unbiased Evaluation Framework for GraphRAG [9], donde se propone un sistema de evaluación reduciendo los sesgos de las métricas utilizadas en otros estudios. Otro trabajo relacionado con el tema es el artículo GraphRAG-Bench: Challenging Domain-Specific Reasoning for Evaluating Graph Retrieval-Augmented Generation [10], donde se propone una solución para evaluar de forma independiente la

construcción del grafo, la recuperación de información, la generación de respuestas y la coherencia del razonamiento.

A partir de estos estudios, herramientas como RAGAS [13] han surgido para intentar evaluar estos componentes con métricas específicas para la calidad del *retrieval*, la fidelidad al contexto recuperado y la detección de alucinaciones.

En este contexto, este trabajo explora varias de estas líneas de investigación. Por un lado, se trata el problema de la construcción automática de grafos de conocimiento a partir de texto no estructurado. Por otro, se estudian formas de recuperación basadas en entidades y relaciones para tareas de pregunta-respuesta multi-salto. Finalmente, se analiza la viabilidad de desplegar este tipo de arquitecturas en entornos locales con recursos limitados, un aspecto todavía poco explorado.

CAPÍTULO 3

Arquitectura propuesta

En este capítulo se describe el trabajo realizado para el desarrollo completo de la arquitectura. Cada sección corresponde a una descripción detallada de las fases introducidas anteriormente y una explicación sobre las herramientas utilizadas y descartadas.

3.1 Descripción general

La arquitectura desarrollada abarca todas las fases necesarias para construir un sistema de recuperación de información basado en grafos de conocimiento. En esta sección se describe el proceso llevado a cabo para el desarrollo de los diferentes módulos que componen el sistema:

- Dataset utilizado.
- Extracción de tripletas a partir de textos no estructurados.
- Construcción de la base de datos utilizada para el almacenamiento del grafo.
- Sistema de recuperación de la información almacenada en el grafo.
- Módulo de generación de respuestas a partir de la información extraída.
- Sistema de evaluación de los resultados.

El flujo de la arquitectura comienza por la extracción de tripletas de los textos proporcionados en el dataset. Una vez extraídas pasan por un proceso de limpieza y normalización antes de ser integradas en la base de datos de grafos.

Cuando el usuario formula una pregunta, esta es procesada por el sistema, donde se extraen las entidades presentes en la pregunta. Con esta información se realiza una búsqueda de nodos similares en el grafo. Una vez detectadas las entidades en el grafo, se recupera un subgrafo, que pasa a ser filtrado para dejar las entidades con relaciones necesarias para contestar a la pregunta. A continuación, estas tripletas se estructuran en un formato apropiado para su procesamiento por el LLM.

Finalmente, el modelo de lenguaje genera una respuesta basada exclusivamente en la información proporcionada. Las respuestas generadas son evaluadas

mediante las métricas definidas con el objetivo de evaluar el rendimiento y la tasa de acierto sobre las preguntas planteadas, así como la efectividad del proceso de recuperación.

Toda la arquitectura ha sido diseñada de forma modular, lo que permite sustituir o modificar componentes y herramientas utilizadas para el refinamiento del sistema.

La Figura 3.1 muestra el flujo completo de la arquitectura:

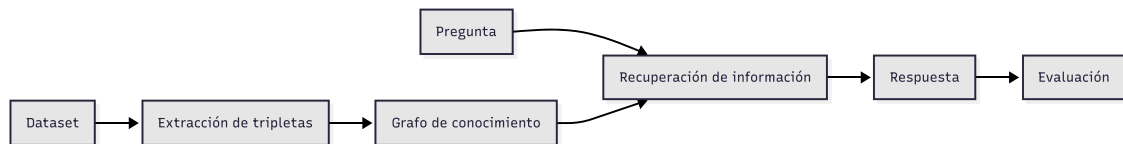


Figura 3.1: Flujo arquitectura

3.2 Selección del dataset

La elección del dataset de trabajo supone la primera decisión relevante del trabajo, ya que determina tanto el proceso de construcción del grafo de conocimiento como la posterior evaluación de los resultados. El requisito principal era buscar un dataset orientado al desarrollo de una base de datos de grafo. Es por esto que inicialmente se consideró la utilización del conjunto de datos AI-KG. Este dataset fue construido a partir de los abstracts de un gran número de papers científicos en el área de la inteligencia artificial, y su misión principal era facilitar la búsqueda de estos artículos mediante palabras clave y relaciones.

A priori, parecía una opción adecuada, ya que integra tanto textos sobre inteligencia artificial, los resúmenes iniciales de los artículos, como las tripletas extraídas de estos. Esto lo convertía en un candidato adecuado, ya que al proveer tanto los textos como las tripletas, aportaba una buena base de referencia sobre la que trabajar. Sin embargo, tras analizar en detalle estas tripletas y su metodología de extracción se vio que su uso no era viable. Las tripletas extraídas no procedían de relaciones explícitas dentro de los textos, sino que habían sido extraídas por una conjunción de tres modelos, donde el proceso comenzaba por extraer todas las entidades relevantes del texto, y después, se conectaban mediante una serie de 27 relaciones predefinidas. Este enfoque era adecuado para el caso de uso sobre el que fue desarrollado, pero no se adaptaba a las necesidades de este trabajo, ya que al no ser relaciones explícitas en el texto, su uso como línea de base lo dificultaba. Se hicieron algunas pruebas de extracción sobre estos artículos donde rápidamente se pudo comprobar que forzar los resultados a coincidir con las tripletas provistas era una tarea alejada del enfoque principal de este trabajo, que se plantea como una extracción general sin esquema predefinido. Además de esto, se observó que estas tripletas habían sido extraídas automáticamente y no todas eran correctas. Según el artículo donde se detalla el proceso, alrededor de un 20 % no eran válidas, por lo que se descartó su uso. Se comenzó entonces otra nueva búsqueda de un conjunto de datos alternativos, con unos requisitos más definidos:

- Disponibilidad de textos a partir de los cuales realizar la extracción de la información.
- Orientado a tareas de extracción y recuperación de información mediante grafos de conocimiento.
- Existencia de una base de referencia para la extracción de conocimiento. Preferiblemente una lista de tripletas ya generadas.
- Aporte de una serie de preguntas con su respuesta correspondiente para utilizar como referencia (ground truth) para una evaluación objetiva.
- Tamaño adecuado para la realización de experimentos en un entorno local con recursos limitados.
- Preferiblemente con uso contrastado y enfocado a la respuesta de preguntas con requerimientos de inferencias multi-salto.

Con estos criterios se analizaron varias alternativas y finalmente se optó por utilizar el dataset ampliamente conocido 2WikiMultihopQA [14], que cumplía todos los requisitos establecidos.

2WikiMultihopQA fue concebido como un benchmark para tareas de pregunta-respuesta multi-salto. Los autores observaron que muchos datasets existentes permitían a la IA deducir la respuesta directamente e inferirla a través de patrones integrados en la pregunta. Con el objetivo de crear unas pruebas más exigentes surgió este dataset. El conjunto de datos se construye a partir de artículos de Wikipedia y contiene preguntas que requieren combinar información procedente de múltiples documentos para obtener la respuesta correcta. Esta característica es especialmente apropiada para este trabajo, ya que permite evaluar la capacidad del sistema para recuperar y relacionar información de diversas áreas, uno de los principales objetivos de las arquitecturas basadas en grafos de conocimiento.

Este dataset incluye más de 160 000 registros en el conjunto de entrenamiento, el utilizado para este desarrollo. Cada uno de los registros contiene:

- Una pregunta en lenguaje natural que requiere inferencia multi-salto.
- Una respuesta (*ground truth*) considerada correcta y utilizada como referencia en la evaluación.
- Un contexto compuesto por varios párrafos extraídos de Wikipedia.
- Una serie de evidencias, que son las entidades y sus relaciones necesarias para contestar a las preguntas. Estas evidencias no están diseñadas para la construcción del grafo, sino para establecer las relaciones necesarias para contestar las preguntas. Sin embargo, al venir en formato sujeto-relación-objeto, se presentan como el escenario perfecto para ser utilizadas como tripletas de referencia.
- Una clasificación por tipo de pregunta. Las preguntas se han agrupado en 3 tipos en función de la tarea que se requiere para obtener la información relevante:

- *Comparison*: requieren una comparación directa entre dos atributos de dos entidades. Ej: ¿Las personas X e Y tienen la misma nacionalidad?
- *Compositional*: requiere seguir varias relaciones entre entidades para encontrar la respuesta. Ej: ¿En qué país nació el director de la película X?
- *Bridge Comparison*: Requiere comparar el atributo de dos entidades distintas no relacionadas y conectar esa información. Ej: ¿El director de que película es de mayor edad, el de X o el de Y?

Para este trabajo se ha optado por trabajar con un subconjunto del dataset. En concreto se evalúan las respuestas de 500 registros. Este tamaño permite obtener resultados suficientemente representativos manteniendo tiempos de ejecución razonables y permitiendo la realización de diferentes experimentos. Entre este subconjunto se ha asegurado la aparición de los 3 tipos de preguntas mencionados anteriormente y se asegura así la evaluación de la capacidad del sistema de obtener la respuesta en función de los diferentes requisitos de la pregunta.

Una de las características más relevantes de 2WikiMultihopQA para este trabajo es la disponibilidad de evidencias (tripletas) asociadas a cada pregunta. Mientras que muchos datasets únicamente proporcionan la respuesta final, este conjunto de datos permite evaluar de forma independiente la calidad de la recuperación de información y la calidad de la respuesta generada. Esto resulta especialmente importante en arquitecturas basadas en grafos de conocimiento, donde el rendimiento del sistema depende tanto de la capacidad para recuperar las relaciones correctas como de la generación final de la respuesta.

Además, en el repositorio del dataset se puede encontrar la implementación de 4 métricas para evaluar la respuesta final. Estas son unas de las métricas clásicas más utilizadas en el ámbito del lenguaje natural:

- Exact Match (EM): Evalúa si la respuesta generada es idéntica a la de referencia.
- Precision: Número de palabras en la respuesta generada que coinciden con la respuesta de referencia.
- Recall: Número de palabras en la respuesta de referencia que se encuentran presentes en la respuesta generada.
- F1 score: media entre precisión y recall.

Estas métricas, aunque ayudan a conocer la fiabilidad y calidad del sistema, no son ni mucho menos suficientes para evaluar el sistema al completo, entre otras cosas porque no evalúan la calidad del *retrieval*, sino únicamente de la respuesta final. Sobre este aspecto, relacionado con la evaluación y las métricas, se hablará más en detalle en la Sección 3.6 dedicada a este tema en concreto.

Por tanto, esta serie de características lo convierten el candidato ideal para la arquitectura al establecer un marco de referencia tanto para la construcción del grafo como para la generación de respuestas a preguntas que requieren varios tipos de inferencias multi-salto.

3.3 Extracción de conocimiento

En este trabajo, la extracción de conocimiento se trata de la extracción de información de los textos para su posterior integración en un grafo de conocimiento. La información se representa en forma de tripletas, las cuales tienen una estructura sujeto-relación-objeto. Con esto se conectan dos entidades mediante una relación identificada directamente en el texto o que se infiere de él. De cada frase es posible extraer varias tripletas, por lo que una entidad puede estar presente en varias de ellas con lo que se consigue que la información quede interconectada. Esto permite hacer asociaciones entre entidades aunque aparezcan alejadas en el texto o incluso que provengan de diversas fuentes.

En la Figura 3.2 se muestra el flujo del proceso llevado a cabo para la construcción del grafo de conocimiento que se explicará en esta sección:



Figura 3.2: Proceso de construcción del grafo de conocimiento

3.3.1. Retos de la extracción

La extracción de conocimiento de los textos para la construcción del grafo constituye probablemente el paso más crítico a la hora de construir la arquitectura. La calidad del grafo depende directamente de la calidad de las entidades y relaciones extraídas a partir de los textos originales. Una extracción incompleta provocará una pérdida de información que impedirá su posterior recuperación. Por otro lado, una extracción de relaciones incorrectas introducirá ruido en el grafo, dificultando o incluso llegando a imposibilitar la extracción de información relevante. Debido a esta dependencia, el rendimiento general del sistema queda limitado por la capacidad de este componente para generar una representación lo más fiel posible al contenido, evitando introducir información irrelevante o errónea al mismo tiempo que se recoge la máxima cantidad de información posible.

La construcción automática de un grafo de conocimiento a partir de textos sin un esquema predefinido presenta varios desafíos importantes:

El primer reto es la detección de entidades, conocida como NER (*named entity recognition*). Una extracción muy restrictiva provocará una falta de contexto general en el grafo, mientras que una extracción permisiva provocará la generación de un elevado número de nodos poco útiles que terminarán por degradar su calidad. Además, la decisión de qué se entiende por entidad es un desafío exigente, ya que es subjetivo en función de las necesidades del caso de uso. Por un lado, se pretende extraer entidades como nombres propios de personas, lugares, etc., pero hay una serie de elementos que pueden ser considerados entidades o no. Un ejemplo ambiguo son los adjetivos, los cuales en algunos contextos pueden ser útiles pero en otros simplemente aportan ruido al grafo. Otro caso pueden ser las fechas, es interesante tener ciertas relaciones temporales si son relevantes, pero

la introducción de muchos nodos de tipo fecha puede suponer una pérdida de calidad en el grafo. Además se necesita un sistema que sea capaz de detectar este tipo de entidades, ya que un número puede ser un año concreto, una cantidad, un identificador, etc.

Una vez extraídas las entidades del texto aparece otro problema conocido como resolución de entidades. Una misma entidad puede aparecer con distintos nombres o estar escrita de diversas formas, ya sea por estar abreviadas o tener diferentes entradas. Así, nombres como "NY", "New York." o "Nueva York" hacen referencia a la misma entidad, sin embargo en el grafo aparecerán como nodos independientes, perdiendo así el sentido de la base de datos de conocimiento donde se pretende tener todo conectado por el menor número posible de nodos. Otro ejemplo claro y aún más preocupante son los nombres propios, que si se representan en nodos separados, cada uno tendrá una información asociada diferente y esta información solo serán accesibles en función de como esté escrito en el momento de la extracción. Encontrar una forma de desambiguación de entidades es también una de las partes más importantes y exigentes, ya que tener nodos que representan lo mismo supone un problema, pero unir en uno solo dos nodos que no representan lo mismo distorsionaría completamente la información contenida.

Otro reto exigente consiste en la identificación de relaciones, problema conocido como '*relation extraction*' (RE). En muchos casos las entidades relacionadas aparecen separadas en distintos párrafos o documentos, o la relación no se encuentra de manera explícita en el texto. Al igual que en la detección de entidades, es un gran reto decidir qué relaciones son relevantes y qué definir como relación. Normalmente la relación se representa con un verbo, pero muchas veces puede ser de otro tipo como relación familiar, profesión, la fecha de un evento o una gran variedad de posibilidades.

La evaluación objetiva de la extracción es otra de las grandes dificultades de esta tarea, y todavía es un problema abierto y discutido. Existe la posibilidad de evaluar la extracción con métricas clásicas como precisión, recall y F1, pero esto solo es posible cuando se dispone de una base de referencia para comparar. En escenarios abiertos, sin esquemas predefinidos se vuelve especialmente complejo el realizar esta evaluación con métricas objetivas.

Por este motivo, en este caso la evaluación de las tripletas obtenidas tras la extracción se realiza de forma experimental, como se verá más adelante, comparando los resultados obtenidos mediante el grafo ideal construido con las tripletas aportadas por el dataset, con los resultados obtenidos mediante el grafo construido automáticamente.

3.3.2. Herramientas de extracción

Para llevar a cabo el proceso de extracción se exploraron varias alternativas. El primer método probado fue a través del uso de modelos de lenguaje para extraer directamente las tripletas en formato sujeto-relación-objeto. Debido a la capacidad de los LLMs de analizar texto, y de la posibilidad de hacerlo sin un esquema predefinido, este es uno de los enfoques más utilizados.

Sin embargo, las pruebas realizadas mostraron resultados muy pobres e insuficientes para la realización del trabajo. Por un lado, una gran parte de las entidades de los textos que no eran detectadas. Por otro lado, las relaciones extraídas eran poco precisas y muy variables. Muchas de ellas eran ambiguas, otras eran excesivamente descriptivas, incluso llegando a ser frases casi completas que contenían otras entidades o no tenían un significado claro. Esto provocaba una gran dificultad para la construcción de un grafo coherente debido a la falta de información, el exceso de ruido y la falta de un significado concreto.

Estos problemas derivan, por un lado, de la complejidad propia de la tarea que ya se ha descrito. Por otro lado, los modelos de lenguaje ejecutados localmente tienen capacidades muy inferiores respecto a las de los grandes modelos utilizados habitualmente para la tarea de extracción, por lo que finalmente se descartó esta posibilidad.

Dado que no existía la posibilidad de realizar la tarea directamente con LLMs, se optó por buscar soluciones ya implementadas y funcionales. Entre estas se estudió la posibilidad de utilizar Microsoft GraphRAG, que realiza el proceso de extracción al completo, desde la detección hasta la desambiguación y agrupación de entidades. Sin embargo, esta solución también delega la tarea en LLMs, y aunque ofrece una pipeline completa de extracción, presentaba limitaciones similares al uso directo de LLMs y el tiempo de procesamiento era extremadamente elevado. Por otro lado, Neo4j, la herramienta de bases de datos de grafos más extendida y la utilizada en este sistema, ofrece una librería propia para la extracción de tripletas que además integra la inserción en la base de datos. Sin embargo, esta herramienta también basa la extracción en el uso de LLMs y funciona mejor con la definición de un esquema, por lo que tampoco se adaptaba a las necesidades del proyecto.

Por estos motivos se decidió explorar modelos diseñados específicamente para la extracción de conocimiento. Tras esta búsqueda, finalmente se optó por utilizar el modelo REBEL (*Relation Extraction By End-to-end Language Generation*). La principal ventaja de REBEL frente a otras alternativas es que genera directamente tripletas estructuradas sin necesidad de ejecutar por separado procesos de reconocimiento de entidades y clasificación de relaciones. Además, al haber sido entrenado específicamente para tareas de extracción, obtiene buenos resultados en distintos benchmarks del área. REBEL no basa su trabajo en otros modelos de lenguaje, sino que es un modelo en sí mismo diseñado específicamente para esta tarea. Este modelo detecta las entidades y sus relaciones de una sola pasada y devuelve las tripletas en un formato `<triplet><subj><obj><rel>`.

El uso de este modelo aportó varias ventajas. Al ser un modelo expresamente diseñado para la extracción de tripletas, consigue una velocidad muy alta de extracción en comparación con el uso de LLMs tradicionales. Utiliza un esquema de relaciones aprendido durante su entrenamiento, lo que evita la creación de relaciones incoherentes. Además, ha sido entrenado con datos extraídos de Wikipedia y Wikidata, lo que lo hace especialmente adecuado para el dataset seleccionado. Este modelo está orientado a utilizar relaciones generales y no permite definir reglas de extracción externas a su entrenamiento. Es por esto que presenta limitaciones en el análisis de textos en un área específica, donde se requiere definir un esquema, pero esto se vuelve una fortaleza en este caso de uso donde se busca un modelo lo más generalista posible.

Una vez elegido se realizaron varias pruebas sobre los textos del dataset que mostraron una mejora significativa respecto a las alternativas evaluadas anteriormente basadas en el uso de LLMs en local. REBEL consiguió detectar una proporción considerablemente elevada de entidades presentes en los textos y generar relaciones consistentes y con sentido.

Con el extractor elegido, se procedió a su configuración. REBEL permite definir 4 parámetros que definen el comportamiento a la hora de realizar la extracción. Estos parámetros fueron elegidos en base a varias pruebas sobre diferentes textos:

- `max_length`: Limita el número máximo de tokens que el modelo puede escribir en su respuesta estructurada. Este parámetro afecta directamente al consumo de memoria RAM o de la tarjeta gráfica en el momento de la generación.
- `num_beams`: Controla cuántas opciones o caminos explora el modelo al mismo tiempo mientras genera texto. Al final de la oración, evalúa cuál de esos caminos tuvo mayor coherencia. Es el parámetro más crítico para la calidad.
- `num_return_sequences`: Establece cuantas variantes diferentes u opciones de respuesta devolver entre las mejores que encontró al terminar su búsqueda. Entre las variantes hay muchas tripletas repetidas, la idea es seleccionar la mejor de estas variantes. Se recomienda poner un valor de 3, pero en este caso se optó por darle un valor más alto, de 7. Una vez generadas, en vez de escoger una, se optó por seleccionar todas, eliminando las duplicadas, para de esta forma recoger la mayor cantidad de información posible.
- `length_penalty`: Modifica la puntuación de las oraciones generadas basándose en su longitud. Cuando el valor es superior a 1 premia las respuestas más largas, cuando es menor favorece respuestas cortas y directas. Se dejó a 0 como viene por defecto, lo que no penaliza ni premia por longitud.

Una vez decididos los parámetros a utilizar se procedió a la extracción de las tripletas sobre los 500 registros utilizados para el desarrollo del sistema.

En el dataset, los textos aparecen por párrafos dentro del contexto. A REBEL se le pasaron estos párrafos que se componen de varias frases simples y que tratan de una entidad en concreto, aunque existe información relacionada que se encuentra en diferentes párrafos, lo que supone uno de los desafíos de este trabajo.

3.4 Construcción del grafo

Con la información extraída en forma de tripletas se pasa a la fase de almacenamiento. Estas tripletas deben almacenarse en una base de datos orientada a grafos, que permita la creación de nodos unidos mediante relaciones, el modelado del grafo y la consulta y extracción de la información.

Para esta tarea se optó por utilizar la base de datos de grafos Neo4j. Esta base de datos se ha establecido como uno de los estándares dentro de la industria, siendo una de las más utilizadas tanto en entornos académicos como industriales

para producción. Su amplio uso hace que sea una herramienta ampliamente contrastada, que dispone de una gran variedad de herramientas internas así como una extensa documentación.

Entre la gran variedad de funcionalidades que ofrece Neo4j [15], hay algunas especialmente útiles para el desarrollo de esta arquitectura. Entre ellas destacan la capacidad de realizar consultas complejas mediante el lenguaje Cypher, lenguaje de consultas utilizado por Neo4j para la consulta de grafos, y la posibilidad de incluir índices tanto para búsquedas semánticas como textuales. Estas herramientas son algunas de las utilizadas en este proyecto y se describen en detalle más adelante, en la Sección 3.5 de *retrieval*.

3.4.1. Modelo de datos

Dentro de Neo4j la información se modela como una estructura basada en nodos, los cuales corresponden a las entidades detectadas en el proceso de extracción, unidos por aristas, las cuales corresponden a las relaciones que conectan las entidades detectadas.

Los nodos de la base de datos pueden tener diferentes propiedades y pueden pertenecer a diferentes clases. Esto permite la construcción de un grafo más ordenado, lo que permite una búsqueda y filtrado entre estas propiedades o tipos. De esta forma, pueden existir nodos de tipo persona, fecha, ubicación, etc. cada uno de ellos con unas propiedades específicas. Sin embargo, aunque es una buena práctica, la identificación y clasificación de las entidades es una tarea compleja, que en muchas ocasiones requiere de un análisis del contexto que rodea a la entidad dentro del texto original. Otra opción, cuando se conocen de antemano los nodos y relaciones que compondrán el grafo, es definir un esquema y clasificar estos nodos dentro de él. Sin embargo, el propósito en este trabajo es utilizar textos de ámbito general, por lo que se optó por no clasificar los nodos en categorías. En este modelo todos los nodos de la red pertenecen a la misma categoría *Entity*, y serán tratados por igual. Esto hace que la elección y el filtrado de la información se delegue en el módulo de recuperación.

Esta decisión tiene otra ventaja y consiste en evitar trabajar con consultas Cypher de complejidad elevada. Si bien este lenguaje permite la búsqueda, el filtrado y la realización operaciones a un nivel de detalle muy preciso, la automatización de estas consultas se vuelve un campo de estudio en sí mismo, especialmente en un entorno sin esquema como el de este trabajo.

De esta forma, el modelo elegido es simple pero permite mantener una representación fiel a los datos, a la vez que aporta flexibilidad a la hora de buscar y extraer la información, sin necesidad de definir un esquema cerrado de entidades y relaciones.

Sobre las relaciones, la filosofía seguida es la misma que la utilizada sobre los nodos. Se ha optado por no clasificarlas, manteniendo únicamente el nombre y la dirección de estas. Conviene destacar que el buen desempeño de REBEL en la detección de relaciones facilita en gran medida este paso, y que mediante el uso de LLMs, sin esquema fijo, no sería posible.

3.4.2. Carga de los datos

Como se ha señalado anteriormente, la mayoría de las librerías relacionadas con el área de trabajo con grafos integran funciones para trabajar con Neo4j. Entre las más utilizadas se encuentran LlamaIndex y LangChain, ampliamente utilizadas por la comunidad. Una de las mayores ventajas de estas herramientas es la abstracción de la comunicación con Neo4j. Estas permiten delegar las tareas respecto a la base de datos en funciones predefinidas. Esto resulta adecuado en muchos casos, pero en otros introduce una serie de problemas debido a la opacidad de estas funciones. En muchas ocasiones, estas funcionalidades son poco flexibles y actúan como caja negra, por lo que se pierde trazabilidad y control sobre la construcción del grafo.

Este problema se acentúa a la hora de consultar y recuperar la información, como se detalla en la Sección 3.5, especialmente en un entorno sin un esquema fijo. Es por esto por lo que se decidió prescindir de estas abstracciones e interactuar directamente con la BBDD mediante consultas Cypher, lo que permite obtener un control completo sobre las operaciones realizadas, tanto en la carga de datos como en la recuperación de la información.

Con estas consideraciones, y utilizando la librería oficial de Neo4j, se cargan las tripletas extraídas de los textos de 500 registros del dataset, construyendo así el grafo de conocimiento del sistema. Para la inserción de las tripletas se hace uso del método MERGE de Neo4j. Este método permite la inserción de nodos y relaciones evitando duplicidades. La ausencia de tripletas duplicadas quedó resuelta en la extracción, ya que se eliminaban las duplicidades antes de crear el set final de inserción. Sin embargo, es una buena práctica introducir los datos con MERGE y no con CREATE, ya que con este último siempre se creará un nuevo nodo o una nueva relación aunque ya exista.

3.4.3. Grafos de referencia

Además del grafo construido a partir de la información extraída de los textos (grafo REBEL), y para establecer una base de referencia, se crean dos grafos adicionales. El primero de ellos se obtiene exclusivamente a partir de las evidencias provistas por el dataset 2WikiMultihopQA. Estas evidencias se tratan como tripletas extraídas y se cargan en Neo4j para la creación de un grafo ideal (grafo de evidencias). En este grafo se encuentra toda la información y solo la información necesaria para contestar a las preguntas planteadas, es por esto que es un buen punto de partida a la hora de comprobar el rendimiento del *retrieval*. Con esto es posible saber cómo se comporta el módulo de recuperación en condiciones ideales, pero no conocer, en condiciones reales, cuantos de los fallos en las respuestas provienen de la ausencia de información debido a la fase de extracción, y cuantos de la pérdida de rendimiento del *retrieval* al usar un grafo real con mucha más información de la estrictamente necesaria. Es por esto que se crea el tercer grafo, como combinación de los otros dos (grafo híbrido). En este tercer grafo de referencia se introduce tanto la información extraída automáticamente con REBEL como las evidencias aportadas por el dataset. Con esto se obtiene un grafo donde se sabe con certeza que la información necesaria para contestar a las preguntas existe, pero está mezclada con la que proviene de la extracción automática. De

esta forma se consigue aislar completamente el rendimiento del *retrieval* del de la extracción.

3.4.4. Análisis comparativo entre grafos

Con los tres grafos construidos independientemente se analizan algunos parámetros básicos para comparar la generación a partir de extracción automática mediante REBEL y el uso de tripletas ideales. En la Tabla 3.1 se puede observar una comparación entre los 3 grafos, donde se aprecia la diferencia en sus dimensiones.

Parámetros grafo	Grafo evidencias	Grafo REBEL	Grafo híbrido (REBEL + evidencias)
Nº de Nodos	1610	12903	13090
Nº de Relaciones	1205	21752	22265
Nº de Relaciones únicas	21	215	215
Grado medio de conexión	1.5	3.36	3.39
Grado máximo	146	440	458
Nº de nodos de grado 1	1155	5156	5231
Nº de nodos aislados	0	0	0

Tabla 3.1: Comparativa entre los grafos de referencia y el construido con REBEL

Con esta comparativa se puede ver la notable diferencia entre los grafos. El nº de nodos y relaciones permite conocer el tamaño de cada grafo. En el caso de la extracción con REBEL se obtiene un grafo con 12903 nodos, más de 8 veces más que el obtenido a través de las evidencias. En el número de relaciones la diferencia es aún mayor ya que en el grafo construido con REBEL encontramos 21752 relaciones, que se dividen en 215 tipos diferentes. Por su parte en el grafo de referencia vemos una diversidad de relaciones aproximadamente 20 veces menor, con 1205 y tan solo 21 tipos diferentes.

Otra métrica interesante para comprobar la magnitud de los grafos es el grado de los nodos, que hace referencia al número de conexiones que tiene cada uno. Así, un nodo de grado 5 implica que tiene 5 relaciones, y presumiblemente 5 nodos conectados, en el caso de que no tenga más de una relación con el mismo vecino. Con el grado medio de los nodos de la red es posible hacerse una idea de cuán conectados están los nodos, lo que implica el grado de interconexión de las entidades extraídas. En la comparativa se puede apreciar cómo los nodos del grafo construido con REBEL tienen una media de 3.3 conexiones mientras que en el de referencia es de menos de la mitad, de 1.5, lo que supone una extracción de mucha menos información por cada entidad.

Por último se comprueba el grado máximo, donde destaca el grafo REBEL, en el que se indica que hay un nodo que tiene hasta 440 conexiones. Estos son los conocidos como supernodos, nodos centrales de la red que aglutinan un gran número de conexiones, y que muchas veces suponen la introducción de ruido a la hora de recuperar la información requerida. La aparición de estos supernodos es una consecuencia habitual de la extracción de conocimiento, ya que determinadas entidades generales como países, años, organizaciones, etc. concentran un gran número de entidades conectados a ellos.

Por último se hacen dos medidas de saneamiento. Por un lado el número de nodos de grado 1, los cuales solo tienen una conexión. Esto no es un problema por

sí solo, ya que hay muchos casos en los que un nodo puede ser una propiedad específica de otro y solo estar relacionado con ese. Sin embargo, un alto porcentaje de nodos de grado 1 implica que aumenta la probabilidad de que aparezcan lo que se conoce como islas de nodos, es decir, grupos de nodos aislados del resto de la red. En términos de calidad del diseño, esto es algo a evitar, ya que lo que se busca con un grafo es tener la información con el mayor grado de interconexión posible. Con esto se comprueba que el grafo de referencia es mucho más disperso, y aunque contiene toda la información que se necesita para contestar a las preguntas del dataset, tiene un 71 % de nodos conectados únicamente a un vecino. Por otro lado, el grafo de uso del proyecto tiene un 39 % de nodos de grado 1, lo que implica una conectividad general mucho mayor.

El número de nodos aislados es un parámetro de saneamiento para comprobar que no han quedado nodos incomunicados, sin ninguna conexión con el resto del grafo, lo que en no aporta información útil y únicamente incrementa el tamaño de la base de datos. Por la forma en la que se ha introducido la información en forma de tripletas este parámetro siempre debe ser 0, ya que todos los nodos están conectados con al menos un vecino.

3.4.5. Normalización y deduplicación

Con los 3 grafos disponibles en Neo4j se realizó una exploración de los datos introducidos. El análisis del grafo evidenció el problema que supone la duplicación de entidades. Debido a que la información proviene de un proceso de extracción automática sobre textos de Wikipedia, muchas de las entidades aparecen representadas de diversas formas ortográficas. Además, se detectaron algunas variaciones ortográficas provenientes de los textos, lo que ocasiona que algunas entidades queden representadas en nodos separados. Para solucionar este desajuste, primero se realiza una normalización básica de las entidades introducidas, pasando todo a minúsculas, eliminando apóstrofes y dobles espacios, entre otras transformaciones. Posteriormente se realiza una deduplicación de las entidades. Este proceso implica la detección de nodos correspondientes a la misma entidad pero descritos con variaciones gramaticales. Esto provoca algunos problemas a la hora de recuperar la información, ya que al detectar una entidad, no será posible extraer todas las conexiones asociadas a ella, sino únicamente a las asociadas al nodo en función de la representación ortográfica en ese caso. Este es un proceso crítico y muy complejo que requiere un equilibrio entre maximizar la cantidad de fusiones de entidades duplicadas y minimizar los falsos positivos. El objetivo es automatizar este proceso, pero esto implica serios riesgos, ya que fusionar nodos que no hacen referencia a la misma entidad (falsos positivos) se traduce en una distorsión de la información y una gran pérdida de calidad. El trabajo de desambiguación se basó, en parte, en una métrica de similaridad aportada por Neo4j que se verá más adelante, por lo que el proceso de agrupación de entidades se explicará en detalle en la Sección 3.5.

3.5 Diseño del retrieval

Una vez construido el grafo de conocimiento, el siguiente paso es el diseño del módulo de recuperación de información o *retrieval*, encargado de extraer la información relevante para contestar a las preguntas planteadas. Este componente es uno de los más críticos de la arquitectura, ya que la calidad y precisión de las respuestas generadas dependen directamente de la capacidad del sistema para encontrar y seleccionar la información adecuada en el grafo.

En los sistemas RAG tradicionales, donde la información recuperada es una parte del texto original, esta búsqueda se realiza mediante similitud semántica. Sin embargo, en un sistema basado en grafos es necesario recuperar nodos y relaciones dentro de un grafo. Esto implica nuevos desafíos como la identificación de entidades y conexiones relevantes así como su posterior filtrado.

Idealmente, una pregunta en lenguaje natural sería traducida automáticamente a una consulta estructurada en el grafo para buscar la información requerida. Sin embargo, la ausencia de un esquema predefinido junto con la variedad y complejidad de las preguntas formuladas dificultan este enfoque.

En esta sección se detallan los problemas encontrados y las soluciones aplicadas para realizar esta tarea. En general, este sistema basa su funcionamiento en la detección de entidades en la pregunta para la recuperación de subgrafos relacionados y su filtrado posterior para construir el contexto necesario con el que poder responder a la pregunta.

3.5.1. Consultas al grafo

La primera parte de la fase de recuperación comienza con la búsqueda de la información en el grafo. Dado que la estructura de esta base de datos se compone de nodos y relaciones, la búsqueda no puede basarse únicamente en similitud semántica, como se hace en los sistemas RAG tradicionales.

Las consultas sobre grafos se realizan mediante el lenguaje Cypher, el cual permite seleccionar nodos y relaciones mediante sus propiedades y realizar operaciones con ellos para su extracción. Para extraer datos de esta base de datos a partir de una pregunta, la forma más directa consiste en traducir la pregunta directamente al lenguaje Cypher, recuperando así la información necesaria para la respuesta. Este enfoque ha sido uno de los más utilizados en los sistemas basados en grafo debido, entre otras cosas, a la capacidad de los LLMs para traducir consultas a este lenguaje desde lenguaje natural a Cypher.

Es por esto que inicialmente se estudió la posibilidad de crear las consultas Cypher automáticamente. Para esta tarea se probó a utilizar el método de LlamaIndex Text2CypherRetriever, diseñado para esta tarea en específico y que además proporciona integración directa con Neo4j. Tras algunas pruebas se comprobó que las consultas generadas eran inconsistentes y fallaban a menudo en consultas relativamente sencillas. También se probó el uso de un modelo especializado en la creación de consultas sobre grafos disponible en la plataforma HuggingFace, el `neo4j/text2cypher-gemma-2-9b-it-finetuned-2024v1`, pero los resultados tampoco eran lo suficientemente consistentes para delegar todas las consultas en

él. También se consideró el generarlas directamente con un LLM en local, con técnicas tanto de zero-shot como de few-shot, pero debido a la necesidad de utilizar modelos relativamente livianos, los resultados tampoco tenían la precisión y consistencia suficientes.

El problema con este enfoque en este caso vuelve a ser la ausencia de un esquema predefinido dentro del grafo. La mayoría de sistemas de generación de consultas Cypher asumen una estructura fija, donde los nodos y relaciones son conocidos de antemano, así como sus propiedades. En este caso tanto nodos como relaciones son extraídos automáticamente a partir de texto no estructurado, lo que dificulta en gran medida la construcción de consultas precisas traducidas del lenguaje.

Además, las preguntas utilizadas en el dataset requieren respuestas multi-salto, donde es necesario transitar entre nodos hasta dar con la entidad o el dato buscado. Esto implica que en muchos casos la entidad requerida para la respuesta no aparezca explícitamente en la pregunta, por lo que la generación automática de consultas se vuelve una tarea especialmente compleja.

Es por estas limitaciones que se decide cambiar el enfoque de búsqueda de información para pasar a un modelo de búsqueda de dos etapas: la detección de entidades en la pregunta y la posterior recuperación de la información a partir de estas. Esto reduce la dependencia de consultas Cypher complejas generadas automáticamente.

3.5.2. Recuperación basada en entidades

La estrategia elegida finalmente para la extracción de información se basa en la extracción de entidades presentes en la pregunta formulada para posteriormente realizar una búsqueda dentro del grafo. La idea es que, independientemente de la cuestión realizada y su complejidad, en esta siempre van a estar presentes una o varias entidades, que servirán de punto de partida para la búsqueda en el grafo. Dado que, en la mayoría de casos, la respuesta estará relacionada con la entidad o entidades presentes en la pregunta, si la información se encuentra en el grafo y este está bien construido, se podrá llegar a la respuesta a partir de la entidad detectada.

Este planteamiento introduce dos desafíos. Por un lado, el hecho de detectar la entidad, lo que no siempre es trivial en función de la pregunta y el contexto. Por otro lado, una vez detectadas, no siempre coinciden exactamente con el nombre del nodo guardado. Durante la fase de extracción estas entidades pueden aparecer representadas con variaciones ortográficas respecto a como aparecen en la pregunta, por lo que una búsqueda exacta por nombre es insuficiente para realizar una recuperación fiable.

Existen varias formas de abordar la tarea de detección de entidades. Por un lado, es posible utilizar un LLM para que la extracción. Sin embargo, debido a que los modelos en local con bajos recursos no son siempre consistentes en cuanto a resultados generados se optó por otra vía. La alternativa escogida fue la de utilizar la ampliamente conocida librería `spacy` [16]. Esta es una librería para Python especializada en el procesamiento del lenguaje natural y se caracteriza por su gran

velocidad de procesamiento y eficiencia. Una de las herramientas principales de esta librería es el reconocimiento de entidades nombradas, conocido como NER, por lo que resulta muy apropiada para el caso de uso. A diferencia de la extracción de conocimiento realizado sobre documentos completos, las preguntas del dataset suelen ser cortas y contienen pocas entidades, lo que permite utilizar modelos especializados en NER con buenos resultados y un coste computacional reducido. Tras realizar algunas pruebas se comprobó la gran eficiencia en el reconocimiento de entidades sobre las preguntas, por lo que esta fue finalmente adoptada.

Con las entidades extraídas surge el problema de encontrarlas dentro del grafo, incluso cuando contienen variaciones ortográficas o errores gramaticales. Para evitar esta limitación se hace uso de los mecanismos de búsqueda que ofrece Neo4j basados en indexación. En concreto se implementa un índice vectorial para la búsqueda por similitud semántica y un índice textual para la búsqueda por similitud textual.

El índice vectorial ofrecido por Neo4j es el `VectorIndex`. Para implementarlo, es necesario generar de antemano una representación vectorial con un modelo de embeddings, no existen restricciones respecto al modelo a utilizar. Una vez generados los embeddings, se cargan en Neo4j como una propiedad del nodo. En el momento en que los nodos tienen asignado su representación vectorial, Neo4j construye automáticamente el `VectorIndex` a partir del tamaño de los vectores generados y la función de similitud. Para este caso se ha optado por utilizar el modelo `bge-small-en-v1.5`, con un tamaño de vector de 384, y la función de similitud coseno. Este modelo ofrece un buen compromiso entre la calidad de los embeddings, la velocidad de procesamiento y el consumo de memoria, aspectos relevantes de esta arquitectura. Para realizar una búsqueda semántica, se calcula el embedding de la entidad a buscar con el mismo modelo utilizado en la creación del índice y Neo4j se encarga de realizar la búsqueda. Esta devuelve una serie de resultados con una puntuación en función del grado de similitud calculado.

En cuanto al índice textual, Neo4j ofrece varios tipos. El utilizado para esta arquitectura es el `FullTextIndex`, el cual es una versión extendida del índice `TextIndex`. Mientras que este último solo permite la búsqueda por coincidencias parciales, cuando la entidad buscada esté contenida en el nombre del nodo, `FullTextIndex` encuentra coincidencias similares aunque no estén contenidas directamente en el nombre. Además de la búsqueda parcial, permite la búsqueda aproximada, lo que se conoce como *fuzzy matching*, basada en métricas de distancias textuales para encontrar palabras parecidas gramaticalmente.

Con los índices creados queda todo preparado para la búsqueda en el grafo de las entidades detectadas. Ninguno de los mecanismos de búsqueda ofreció resultados lo suficientemente sólidos por sí solo. La búsqueda exacta ofrece una precisión muy elevada pero un *recall* muy bajo, mientras que las búsquedas aproximadas devuelven un gran número de resultados pero a costa de introducir falsos positivos. Es por esto que se emplea una estrategia híbrida, donde se combinan los resultados de varias búsquedas para encontrar los nodos más similares a las entidades detectadas. Esta combinación agrupa:

- Una búsqueda exacta. Lo primero es buscar si la entidad detectada se encuentra en uno de los nodos del grafo. En caso afirmativo queda claro que es el nodo que se está buscando.

- Una búsqueda por similitud semántica mediante `VectorIndex`. Se seleccionan los dos resultados con mayor puntuación, siempre que pasen un umbral definido.
- Búsqueda parcial mediante `FullTextIndex`. Se buscan nodos que contengan una parte igual que la entidad detectada. Se descartan los que no superen un umbral definido en la puntuación devuelta.
- Búsqueda aproximada (*fuzzy matching*) mediante `FullTextIndex`. Se realiza una búsqueda de nodos que se parezcan textualmente a la entidad, sin necesidad de que contengan una parte exactamente igual. Esta última búsqueda se implementa por los casos extremos en los que ninguna de las otras búsquedas da resultado. Al igual que en el resto, se descartan aquellos en los que la puntuación no supere el umbral establecido.

Con estos cuatro resultados, por un lado se seleccionan los obtenidos en la búsqueda exacta, que se seleccionan directamente. Por otro lado, se presupone que los nodos recuperados por diferentes mecanismos tienen una mayor probabilidad de corresponder a la entidad correcta. Por esto, se filtran los candidatos con un mayor número de apariciones en la unión de las 4 búsquedas. De estos se seleccionan finalmente los 3 candidatos que más veces aparecen y finaliza la búsqueda de entidades dentro del grafo.

3.5.3. Deduplicación

Durante el desarrollo de la fase de búsqueda de entidades en el grafo apareció el problema de la duplicación de entidades antes mencionado. Se observó que al realizar las búsquedas de entidades, estas aparecían duplicadas en diferentes nodos. El proceso de extracción automático genera variaciones en la escritura de los nombres, ya sea por abreviaturas, errores gramaticales o pequeñas variaciones en la detección de la entidad. Esto provoca una fragmentación en el grafo que limita enormemente su rendimiento.

Al analizar los resultados obtenidos por la búsqueda mediante el índice textual `FullTextIndex` de Neo4j se observó que este es capaz de detectar correctamente entidades similares, aún cuando existían variaciones importantes en su escritura. Por esta razón se valoró la posibilidad de utilizar estas puntuaciones para la desambiguación de entidades y poder fusionar los nodos que hacen referencia a la misma entidad.

Sin embargo, esta puntuación no es suficiente para afirmar que una entidad está duplicada, ya que en muchas ocasiones, nodos muy similares gramaticalmente no hacen referencia a la misma entidad. Es por esto que se decidió investigar el funcionamiento interno del índice textual de Neo4j para implementarlo de forma independiente y tener más control a la hora de fusionar los nodos.

Neo4j utiliza como base de su índice el algoritmo BM25, una métrica muy utilizada en sistemas de búsqueda y recuperación de información. Esta métrica permite establecer el grado de similitud entre dos textos teniendo en cuenta la frecuencia de distribución de los términos que componen estos textos. Existen dos librerías diseñadas especialmente para este uso, la `rank-bm25`, que implementa

el algoritmo de búsqueda, y una versión mejorada, la *bm25s* [17], que está optimizada para ser mucho más rápida y además incorpora algunas funcionalidades adicionales como la tokenización, lo que con la versión inicial *rank-bm25* hay que hacer manualmente. Es por esto que se decide implementar el uso de *bm25s* para calcular la similitud entre los nodos del grafo.

Pero como se indicaba con anterioridad, esta métrica no es suficiente, ya que hay una gran cantidad de pares de nodos que reciben una puntuación muy alta pero que no hacen referencia a la misma entidad. Por otro lado hay casos en los que ocurre lo contrario, dos nodos que corresponden a la misma entidad obtienen puntuaciones relativamente bajas.

Debido a esta limitación se decide complementar esta información con la introducción de cuatro métricas de similitud textual que ayudan a detectar diferencias y similitudes gramaticales. Cada una de estas métricas ayuda a encontrar similitudes entredostextos a partir de distintas medidas. Las métricas utilizadas son:

- *Levenshtein*: mide el número mínimo de inserciones o sustituciones necesarias para transformar una cadena de texto en otra. Muy útil para detectar fallos de escritura.
- *Jaro-Winkler*: mide la similitud entre dos cadenas teniendo en cuenta caracteres coincidentes y transposiciones. Útil para cuando la entidad contiene varias palabras que se encuentran en distinto orden.
- *Jaccard*: Mide la proporción de los tokens compartidos respecto al total en ambas cadenas de texto.
- *Overlap Coefficient*: Mide el grado de coincidencia entre dos cadenas de texto en relación a la de menor tamaño. Útil cuando una entidad está contenida dentro de otra.

Con la combinación de estas 4 métricas más la puntuación obtenida mediante *bm25s*, ya es posible determinar con un alto grado de fiabilidad que nodos hacen referencia a la misma entidad para proceder a su fusión. Para no sobrepasar los límites computacionales mediante el cálculo de métricas, se extrajo, en primer lugar, la puntuación mediante *bm25s* entre todos los nodos del grafo. Una vez obtenidas, se escogen los tres candidatos con mayor puntuación para calcular el resto de métricas. Con los resultados de todos los cálculos se establecen una serie de filtros, cuyos valores se decidieron experimentalmente, para obtener finalmente una lista de nodos duplicados.

El análisis manual de los resultados permitió detectar casos de falsos positivos que pasaban todos los umbrales sin ser la misma entidad. La mayoría de estos casos compartían las mismas propiedades. Ejemplos claros son entidades numeradas con números romanos (Fernando VI y Fernando VII) que son muy similares pero claramente no hacen referencia a la misma entidad. Entidades numeradas (27° certamen de música, 28° certamen de música) o nombres de familiares que comparten gran parte de la cadena de texto, especialmente nombres provenientes del árabe, son algunos de los casos que dificultaban asegurar la duplicación de entidades. Por esto se establecieron unos criterios de exclusión. Mediante expresiones regulares (*regex*), se detecta si las diferencias entre ambas entidades

corresponden únicamente a números naturales, números romanos o nombres propios, descartando estos casos de la deduplicación.

Tras la exclusión de estos casos del proceso, quedan seleccionados los candidatos finales a fusionar. En este punto es necesario realizar un proceso de agrupación, ya que una misma entidad puede aparecer representada por más de dos nodos diferentes, y el objetivo es unificar todos los nodos que representen una misma entidad.

Por último, es necesario decidir qué nodo permanecerá en el grafo, es decir, qué nombre se utilizará finalmente para determinar como se llama realmente la entidad. Con el objetivo de mitigar los efectos de la fusión de falsos positivos, se optó por dejar como nombre de cada grupo aquel cuyo nodo en el grafo tenga un grado mayor. Esto presupone que los nodos con mayor número de conexiones son menos propensos a contener fallos gramaticales, y en el caso de obtener un falso positivo, las consecuencias de eliminar un nodo con mayor número de relaciones son mayores que con uno que tenga un grado menor.

Por último, esta lista con los grupos a fusionar se introduce en Neo4j, que mediante el método `mergeNodes` fusiona todos los nodos del grupo en uno solo, manteniendo el nombre del que tenga un mayor número de relaciones. El resto de nombres se guardan como una propiedad de Neo4j para conservar la trazabilidad de la deduplicación.

Al finalizar el proceso se detecta un total de 1449 nodos identificados como duplicados, que tras la agrupación quedan representados por 661 entidades. Esto supone una reducción de 788 nodos, un 6 % del número total en el grafo. Cabe destacar que este no es el número real de duplicaciones. Para no distorsionar la información del grafo se ha seguido una estrategia muy conservadora, empleando umbrales muy restrictivos para minimizar al máximo el número de falsos positivos. En un entorno controlado sería una recomendable realizar una revisión posterior de los candidatos que no han sido fusionados. Pudiendo, a través de las métricas, validar manualmente algunos candidatos que no han pasado los filtros pero que quedan cerca de los umbrales establecidos y que deben ser deduplicados.

Con esta fase finaliza la preparación y construcción del grafo de conocimiento. A partir de este punto, la base de datos queda preparada para iniciar la fase de recuperación de la información.

3.5.4. Recuperación y filtrado de información

Con las entidades de la pregunta identificadas y localizados sus nodos correspondientes dentro del grafo comienza la fase de recuperación de la información relevante. El desafío de este proceso consiste en encontrar un equilibrio entre recuperar suficiente información para la pregunta planteada introduciendo la menor cantidad de información no relevante para el caso, lo que solo aporta ruido a la hora de generar la respuesta final.

Dado que la estrategia seguida es rescatar la información a partir de unos nodos elegidos, este sistema recupera subgrafos a partir de estas entidades de entrada. Concretamente, para cada uno de los nodos originales, se recuperan todos los caminos junto con sus nodos vecinos con hasta dos saltos de profundidad,

es decir, los nodos vecinos directos del original y los nodos vecinos de segundo grado, para los que hay que realizar dos saltos desde el original. Junto a estos nodos también se devuelven las relaciones que los unen, así como sus direcciones. Otra propiedad recuperada es el grado de todos los nodos, tanto del original como de los intermedios, información que posteriormente sirve para el filtrado de la información.

Se escoge la recuperación de subgrafos con profundidad de hasta dos saltos desde el nodo original por un compromiso entre cantidad de información disponible y ruido. Una recuperación limitada a un único salto únicamente permite conocer la información asociada directamente al nodo original. Esto eliminaría por completo la fortaleza del grafo, que es la de conectar información de entidades separadas, y no ofrecería ninguna ventaja respecto al uso de una búsqueda semántica tradicional sobre los textos originales.

Por otro lado, aumentar la profundidad a tres o más saltos supone un crecimiento prácticamente exponencial en la cantidad de nodos y relaciones recuperados. Esto introduce una enorme cantidad de información irrelevante y dificulta el proceso de selección de las relaciones realmente útiles. Además, para este caso de uso, el dataset 2WikiMultihopQA está diseñado específicamente con preguntas que, en su mayoría, pueden ser resueltas con inferencia de dos saltos.

Durante las pruebas realizadas se comprobó que realmente la mayoría de las evidencias necesarias para responder a las preguntas se podían encontrar con un máximo de dos saltos desde el nodo inicial. Por estas razones la recuperación de subgrafos con vecinos indirectos de segundo nivel resulta un compromiso adecuado.

Una vez extraídos los subgrafos correspondientes a los nodos encontrados en la primera fase del *retrieval* se hace un primer filtrado de información basado únicamente en la estructura del grafo. La cantidad de información recuperada en primera instancia es muy elevada, con casos donde se extraen más de 1000 caminos diferentes. Esto implica la presencia de una gran cantidad de información irrelevante que debe ser descartada antes de continuar con el proceso.

En primer lugar se eliminan aquellos caminos con dos saltos donde el nodo intermedio presenta un grado superior a un umbral definido para descartar los caminos cuyo nodo intermedio es un supernodo. En la práctica, los caminos que atraviesan estos nodos rara vez aportan información útil para responder a una pregunta concreta y solo introducen ruido al sistema, por lo que estos caminos son descartados.

Otros caminos que se ha visto que son irrelevantes en la gran mayoría de casos son aquellos con dos saltos en los que ambas relaciones son las mismas y presentan un patrón en el que la dirección de la primera es de salida y la segunda es de entrada. Este patrón aparece normalmente cuando un nodo se conecta a una categoría o atributo común al que están conectados otros nodos que nada tienen que ver con el original. Un ejemplo claro es el de nacionalidad. Si el nodo original está conectado a su lugar de origen, este segundo nodo de tipo ubicación tendrá relaciones con un gran número de entidades con escasa o nula relación con la original.

Con este cribado inicial se consigue eliminar gran parte de caminos irrelevantes, dejando el proceso preparado para las siguientes fases donde se deben identificar los caminos realmente útiles. Pero, aún con el primer filtrado, sigue quedando un elevado número de información irrelevante para la pregunta planteada.

Para la selección de la información a partir de los caminos se emplea un sistema de *reranking*, donde estos caminos son comparados por pares junto con la pregunta original. Inicialmente, las cadenas de texto se construyeron uniendo los nodos y relaciones mediante flechas. Este enfoque presentaba resultados aceptables en algunos casos, pero en muchos otros ofrecía resultados muy pobres, asignando puntuaciones muy bajas a los caminos correctos.

Tras analizar los resultados se comprobó que los modelos de *reranking* ofrecen mejores resultados con inputs de texto en lenguaje natural que con estructuras basadas en símbolos como flechas. Es por esto que se cambió el formateo de unión de los nodos y relaciones mediante flechas por una regla sencilla, donde los nodos y la relación se unen mediante las palabras *which* e *is*. Este cambio provoca que el texto introducido sea más similar a los datos con los que se entrenó el *reranker*, ya que estos modelos se entrenan con textos en lenguaje natural. Esto permitió mejorar notablemente los resultados del *reranking*, ofreciendo resultados más consistentes al asignar puntuaciones bajas a caminos descartables y no penalizar en exceso los caminos correctos.

Sin embargo, con este reranqueo de los caminos completos junto con la pregunta sigue existiendo un problema importante. Las entidades presentes en la pregunta aparecen también en los caminos recuperados, lo que provoca que muchas de las puntuaciones elevadas se deban principalmente a la coincidencia de estas entidades. Esto por un lado aporta el beneficio de filtrar los caminos con entidades detectadas erróneamente. Pero por otro lado provoca que las puntuaciones del *reranker* estén muy asociadas a la entidad y no tanto a la relación, con lo que se pierde la capacidad de distinguir que camino dentro de cada entidad es el más relevante para el caso.

Para mitigar este efecto se introdujo una segunda fase de *reranking*. Esta vez se evalúa la similitud entre la pregunta y únicamente las relaciones de los caminos recuperados de forma aislada, eliminando las entidades recuperadas de la comparación. De esta forma se consigue medir específicamente la relevancia de la relación respecto a la pregunta, lo que ayuda enormemente a seleccionar los caminos correctos.

Aunque este segundo proceso de *reranking* considerando únicamente únicamente las relaciones es muy útil para conocer la relevancia de un camino eliminando la dependencia de las entidades, combinar ambas puntuaciones directamente es inviable. En este segundo *reranking* de relaciones, al no haber coincidencia exacta de términos, las puntuaciones obtenidas son mucho más bajas que las obtenidas con la comparación del camino entero, de un orden de magnitud entre 2 y 4 veces inferior, por lo que una ponderación opacaría la aportación de este segundo valor. Es por esto que se aplica una normalización a las puntuaciones de este segundo *reranker*, mediante una transformación logarítmica en un intervalo $[0, 1]$ para poder realizar una ponderación más precisa.

Con las dos puntuaciones de los *rerankers* en una misma escala se procede a hacer una suma ponderada de ambas, donde se elige a cual de los *rerankings* darle

más peso y en que proporción. Este es un parámetro opcional del sistema, que tras varias pruebas se establece en un 0.65 de peso para el *reranking* del camino completo y un 0.35 para la puntuación obtenida a partir de la comparación de solo relaciones. Esta combinación permite tener en cuenta al mismo tiempo la relevancia de la presencia de las entidades y la relevancia de las relaciones que las unen.

Con las puntuaciones calculadas se procede a la selección final de los candidatos. Para esto, primero se establece un filtro, donde se eliminan todos aquellos caminos que no superan un umbral mínimo, el cual es un parámetro configurable. Para el sistema final este se ha establecido en un valor de 0.4, pero esto dependerá en gran medida del *reranker* utilizado, ya que cada modelo de *reranking* utiliza su propio sistema de puntuación.

Sobre los candidatos que pasan el filtro se seleccionan finalmente los tres caminos con mayores puntuaciones para cada entidad inicial, que son los que en última instancia pasan al LLM para contestar a la pregunta planteada. La elección de escoger los 3 caminos con mayor puntuación de cada entidad se debe a la comprobación experimental del resultado de los *rerankers*. Aquí se comprobó que con un *reranker* adecuado, los caminos correctos quedan, en la gran mayoría de casos, en las primeras posiciones. Es por esto, que con la selección de los 3 primeros candidatos se consigue una alta probabilidad de recuperar la información correcta mientras que se limita la cantidad de información devuelta reduciendo la introducción de información irrelevante y ruido.

Con esto se finaliza el módulo de recuperación y la información queda preparada para su envío al LLM. En la Figura 3.3 se muestra un diagrama del proceso completo de recuperación de la información.

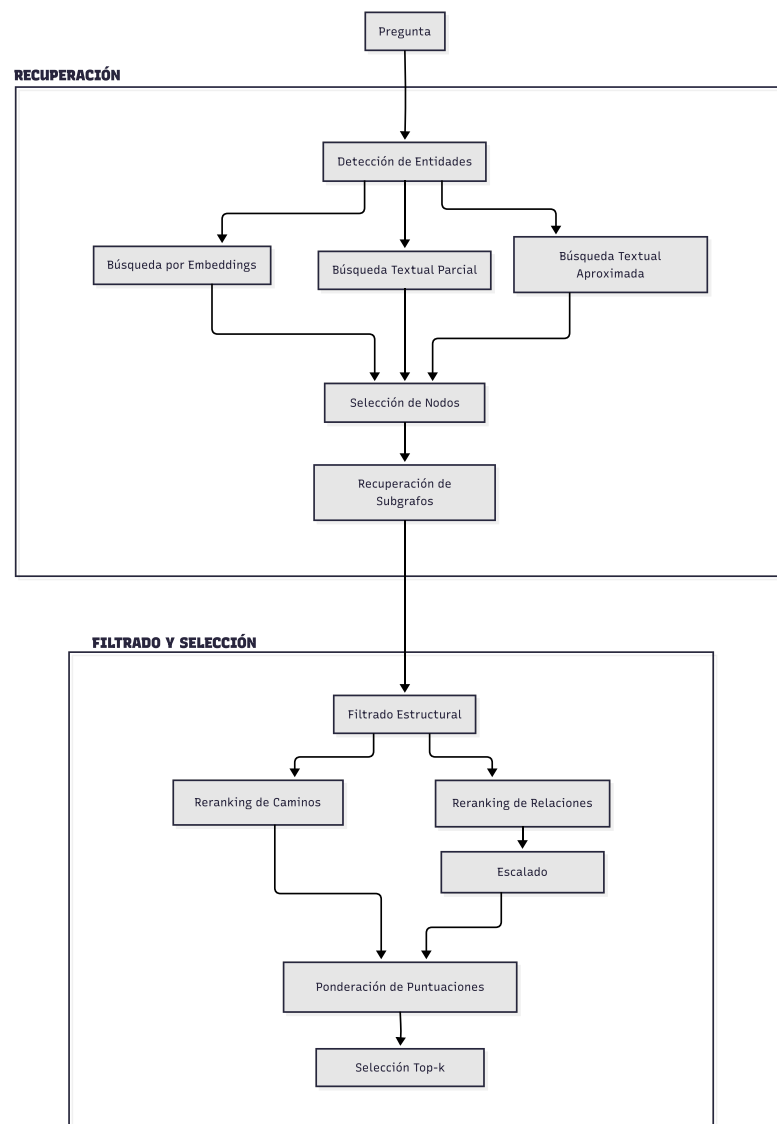


Figura 3.3: Proceso de recuperación de la información

3.5.5. Generación de la respuesta

Con los caminos más relevantes seleccionados, donde se espera que se encuentre la información solicitada por la pregunta, el último paso del sistema es generar la respuesta en lenguaje natural. Para ello se utiliza un modelo de lenguaje al que se le proporciona un prompt que contiene la pregunta original, el contexto recuperado por el *retrieval* y unas instrucciones, donde se le indica al modelo que debe contestar utilizando exclusivamente la información proporcionada por el *retrieval* entre otros requerimientos.

Los caminos finalmente seleccionados se introducen en el modelo con la misma transformación que se utilizó para calcular su puntuación con el *reranker*. Como se comprobó anteriormente, una representación en lenguaje natural se interpreta de forma más efectiva por los modelos de lenguaje que una representación basada en símbolos como flechas o guiones al ser un lenguaje más cercano a los datos de entrenamiento de los propios modelos.

Para esta tarea de generación de la respuesta se han evaluado diferentes modelos de lenguaje ejecutados localmente con diferentes configuraciones en el prompting. La selección de modelos se realizó con el objetivo de evaluar el rendimiento de los modelos en local teniendo en cuenta las limitaciones en los recursos. Lo que se busca es encontrar un equilibrio entre calidad de respuestas y coste computacional.

Sobre el diseño del prompt se construyó teniendo en cuenta algunas consideraciones respecto al dataset 2WikiMultiHopQA. En este, las respuestas de referencia proporcionadas son muy concisas y contienen únicamente las palabras estrictamente necesarias para contestar a la pregunta. En muchos casos esta respuesta se limita simplemente a contestar sí/no, en otros casos la respuesta contendrá exclusivamente el dato solicitado.

Estas características tienen un impacto directo sobre las métricas oficiales del dataset. En estas métricas se penalizan las explicaciones adicionales, otorgando una puntuación de 0 tanto en EM como en precisión, recall y F1 si la respuesta de referencia es de sí/no y la generada no es exactamente igual. Por este motivo se evaluaron diferentes estrategias de *prompting* para reducir estas explicaciones y acotar al modelo a responder lo estrictamente necesario. El objetivo por tanto no es únicamente contestar a la pregunta correctamente, sino hacerlo en un formato estricto y compatible con los criterios de evaluación del dataset.

Una de las instrucciones más importantes incluidas en el prompt consiste en indicar explícitamente al modelo que responda *I don't know* cuando la información necesaria para responder no se encuentre disponible en el contexto recuperado. Esta restricción busca reducir las alucinaciones y permite diferenciar entre errores del proceso de recuperación y errores propios del modelo de lenguaje.

Cabe destacar que el modelo de lenguaje utilizado no tiene acceso directo al grafo de conocimiento ni al proceso de *retrieval*. Su función está limitada a interpretar la información seleccionada en el proceso de recuperación y generar una respuesta teniendo en cuenta únicamente la aportación de la información proporcionada. Esta separación permite evaluar de forma independiente la calidad del sistema de recuperación y la del sistema de generación de respuesta.

Esta necesidad de medir ambos sistemas introduce, por otro lado un problema de evaluación. Con las métricas oficiales aportadas en el dataset solo es posible medir la calidad de la respuesta final. Esto provoca que no sea posible evaluar muchos aspectos de la calidad del resto del sistema, como la calidad del proceso de recuperación, u otros aspectos de la generación de la respuesta, como el grado de alucinación del modelo. Además, al ser unas métricas basadas en el recuento y coincidencia de tokens, se pierde información sobre la calidad real de la respuesta y del sistema en general. Es por esto que se desarrollan métricas complementarias, orientadas a evaluar de manera más precisa los componentes del sistema. Estas métricas se describen en la siguiente sección.

3.6 Métricas de evaluación

La evaluación supone una de las fases más importantes de la arquitectura, ya que permite conocer tanto la calidad de las respuestas generadas como el comportamiento de los distintos componentes que intervienen en el proceso. En esta sección se analizan las limitaciones de las métricas oficiales del dataset y se proponen métricas adicionales para evaluar de forma independiente el módulo de recuperación y el de generación de respuestas.

3.6.1. Limitaciones de las métricas oficiales

El dataset 2WikiMultihopQA incorpora una serie de métricas ampliamente utilizadas en tareas de pregunta-respuesta: Exact Match (EM), Precision, Recall y F1. Estas métricas permiten comparar la respuesta generada por el sistema con la respuesta de referencia proporcionada por el dataset, ofreciendo una medida objetiva de su calidad.

Sin embargo, estas métricas presentan una limitación importante para el análisis de arquitecturas integrales como la de este trabajo. Todas ellas evalúan exclusivamente la respuesta final generada, sin medir de ninguna manera el comportamiento interno de los distintos componentes del sistema.

Esta limitación impide localizar el origen de los errores producidos. Una respuesta incorrecta puede deberse tanto a una recuperación insuficiente de información por el módulo de *retrieval* como a una interpretación errónea o alucinación en la generación de la respuesta por parte del modelo de lenguaje. De igual manera, una respuesta correcta no garantiza necesariamente que el *retrieval* haya funcionado correctamente, ya que el modelo puede no haber seguido las instrucciones de manera estricta y haber llegado a la respuesta por conocimiento adquirido durante su entrenamiento.

Por otro lado, el cálculo de las métricas oficiales del dataset penaliza en gran medida las explicaciones. Si la respuesta de referencia es sí/no, toda respuesta que no coincida exactamente se penaliza con un 0 en las cuatro métricas oficiales. Por lo que si la respuesta ofrecida por el modelo es correcta pero incluye alguna explicación se contabiliza como incorrecta, evaluando con un 0 todas las métricas.

Por otro lado, durante la fase de generación se fuerza al modelo, mediante las instrucciones en el *prompt*, a responder con *I don't know* cuando la información proporcionada por el *retrieval* no contiene la información necesaria o está incompleta. Esta medida busca reducir las alucinaciones y forzar al modelo a basar sus respuestas únicamente en la información proporcionada. Las métricas oficiales tampoco permiten diferenciar los casos en los que realmente la información necesaria no está presente o los casos donde el modelo de lenguaje no ha sabido interpretar correctamente la información que sí estaba presente.

3.6.2. Métricas adicionales

Debido a las limitaciones de las métricas ofrecidas por el dataset y con el objetivo de comprender mejor el comportamiento del sistema se consideró necesario incorporar métricas adicionales que permitan evaluar de forma independiente el módulo de *retrieval* y el de generación de respuestas. En una arquitectura RAG-KAG, recuperar la información correcta es un requisito imprescindible para obtener una respuesta correcta, por lo que comprobar si dicha información ha sido efectivamente recuperada es fundamental para conocer la calidad del sistema.

Otro objetivo era evaluar el comportamiento del modelo de lenguaje una vez proporcionado el contexto. Es necesario poder distinguir entre respuestas incorrectas debidas al *retrieval* y respuestas incorrectas producidas por errores de interpretación o generación. De igual forma, se buscó detectar situaciones en las que el modelo genera información que no aparece en el contexto recuperado (alucinaciones).

También se consideró relevante identificar los casos de evasión incorrecta. Estos casos ocurren cuando el modelo responde con *I don't know* a pesar de que la información necesaria para responder se encuentra presente en el contexto recuperado. Dado que esta respuesta forma parte de la estrategia seguida para reducir alucinaciones, es importante poder medir hasta qué punto el modelo la utiliza correctamente.

Por último, se estudió la manera de medir la precisión del *retrieval* evaluando qué proporción de los caminos recuperados resultaban realmente útiles para responder a la pregunta. Sin embargo, durante las pruebas, se observó que esta métrica es muy subjetiva, incluso para los humanos, por lo que diferentes evaluadores podían llegar a conclusiones diferentes sobre el grado de relevancia de un mismo camino. Además, esta medida aporta menos información sobre el comportamiento del sistema, que conocer si el contexto recuperado resulta suficiente para responder correctamente a la pregunta. Por este motivo se decidió descartar esta métrica.

3.6.3. Definición de las métricas

Con estos requisitos de medición se decide implementar cuatro métricas adicionales a las del dataset para medir estos parámetros.

- **Answer correctness:** Evalúa si la respuesta es correcta, sin importar las explicaciones dadas por el modelo ni la información aportada en el contexto. Útil

para conocer el número de respuestas correctas independientemente de si los tokens coinciden exactamente con la respuesta de referencia.

- **Hallucination:** Evalúa si la respuesta generada por el modelo de lenguaje se encuentra realmente en el contexto aportado por la fase de recuperación de la información o si, por el contrario se ha inventado la respuesta. Esta métrica debe ser 1 cuando se considere alucinación y 0 cuando la respuesta realmente esté completamente respaldada por los datos. En los casos donde el modelo responde con evasión (*I don't know*) se excluyen de ser considerados como alucinación, ya que aunque sí sea posible encontrar la respuesta en los datos no se considera que el modelo haya inventado la respuesta.
- **Context recall:** Mide si la información recuperada es suficiente para contestar a la pregunta planteada. Se plantea como una métrica estricta, donde toda la información debe estar explícitamente contenida en los datos, sin que haya que realizar ninguna suposición o inferencia fuera de ellos, independientemente de que el modelo de lenguaje sea capaz de utilizarla.
- **Incorrect evasion:** Evalúa si se ha producido una evasión incorrecta. Se considera evasión incorrecta cuando el sistema responde con la evasiva *I don't know* teniendo la suficiente información para contestar a la pregunta.

Estas cuatro métricas se obtienen directamente de las respuestas generadas por el sistema. Con estas ya es posible evaluar la calidad del módulo de recuperación así como el de la generación de la respuesta sin depender de la coincidencia exacta de tokens. Sin embargo, con el objetivo de comprender mejor la calidad de la salida final y el rendimiento del sistema general se crean otras tres métricas que surgen de la combinación de las cuatro anteriores:

- **Correctness hallucination:** Identifica los casos donde la respuesta ha sido correcta pero no está respaldada por el contexto recuperado. Se obtiene de los casos donde `answer_correctness=1` y `hallucination=1` y con esto podemos tener una mejor visión de la fiabilidad del sistema.
- **Missed retrieval:** Métrica diseñada para conocer los casos donde el *retrieval* proporciona la información suficiente para contestar a la pregunta pero la respuesta del sistema no es correcta, tanto por evasión como por respuesta incorrecta. Esta proviene de los casos donde `answer_correctness=0` y `context_recall=1` y es útil para medir el rendimiento del LLM utilizado y su capacidad para comprender y utilizar la información proporcionada. Con esto se descartan fallos del sistema y se atribuyen al LLM.
- **Correctness supported:** A diferencia de las dos anteriores, esta métrica contabiliza los casos en los que el sistema al completo ha funcionado correctamente. Se obtiene de los casos donde `answer_correctness=1` y `context_recall=1` e indica los casos en los que realmente la respuesta ha sido correcta y a su vez respaldada por la información extraída por el *retrieval*. Da una medida de los casos donde el sistema ha funcionado correctamente en su totalidad.

Con estas siete métricas, junto con las 4 aportadas por el propio dataset, se dispone de un conjunto de métricas capaces de evaluar con mayor precisión el

comportamiento de los distintos componentes. Es posible conocer si el módulo de *retrieval* recuperó la información correcta, si una respuesta incorrecta es culpa de la información proveída o del propio modelo, o si por el contrario una respuesta correcta está respaldada por la información del *retrieval* o se trata de una alucinación.

Métrica	Retrieval	LLM	Sistema
Exact Match (EM)			Sí
Precision			Sí
Recall			Sí
F1			Sí
Answer Correctness		Sí	Sí
Context Recall	Sí		Sí
Hallucination		Sí	Sí
Incorrect Evasion		Sí	Sí
Correctness Supported	Sí	Sí	Sí
Missed Retrieval	Sí	Sí	Sí
Correctness Hallucination	Sí	Sí	Sí

Tabla 3.2: Componentes del sistema evaluados por cada una de las métricas utilizadas.

Con las métricas definidas, el desafío es obtenerlas de manera automática y fiable. Para esta tarea es necesario acudir a un evaluador externo, que sea independiente del sistema para evitar sesgos y obtener respuestas con un intervalo de confianza elevado, ya que una evaluación incorrecta del sistema impide conocer el estado real de funcionamiento y por tanto su refinamiento.

Se muestra a continuación un resumen con las métricas utilizadas y su descripción:

Métrica	Aspecto evaluado
Exact Match	Coincidencia exacta con la respuesta de referencia.
Precision / Recall / F1	Similitud léxica entre la respuesta generada y la de referencia.
Answer Correctness	Corrección semántica de la respuesta.
Context Recall	Si el contexto recuperado contiene la información necesaria.
Hallucination	Si la respuesta está respaldada por el contexto recuperado.
Incorrect Evasion	Uso incorrecto de la respuesta "I don't know".
Correctness Supported	Respuestas correctas sustentadas por el <i>retrieval</i> .
Missed Retrieval	Casos donde el contexto era suficiente pero el LLM falló.
Correctness Hallucination	Respuestas correctas obtenidas mediante alucinación.

Tabla 3.3: Resumen de las métricas utilizadas y su definición.

3.6.4. Evaluador externo

Para poder evaluar la fiabilidad de las métricas propuestas fue necesario disponer de una referencia independiente sobre la que comparar los resultados obtenidos.

Inicialmente se consideró el uso de herramientas especializadas en este tipo de evaluación de sistemas RAG. Una de las más ampliamente utilizadas dentro de la industria es RAGAS, que automatiza la evaluación y ya integra varias de las métricas con las que se quiere trabajar, lo que lo situaba como una solución idónea. Sin embargo, esta herramienta tiene un coste computacional muy elevado. Para

el cálculo de estas métricas se realiza un gran número de llamadas al LLM para poder hacer comparativas. Esto provoca comportamientos erráticos al ejecutar en un entorno local con recursos limitados. Las métricas a veces no se generaban o eran inconsistentes, además, con la evaluación de una cantidad elevada de casos, el tiempo invertido era inasumible. Por estas razones se descartó su uso.

Dado que otras herramientas especializadas en la evaluación de sistemas RAG tienen funcionamientos similares, se optó por prescindir de ellas e intentar obtenerlas por otro medio. Ante esta situación se optó por implementar un sistema basado en la estrategia conocida como *LLM-as-a-Judge*, donde un segundo modelo de lenguaje actúa como evaluador del sistema principal. Este enfoque es similar al de herramientas como RAGAS, pero al implementar un evaluador propio es posible tener un control sobre los recursos consumidos que de otra forma es imposible. El uso de un LLM como juez permite formular preguntas específicas sobre el comportamiento del sistema y obtener respuestas estructuradas para cada una de las métricas definidas previamente.

El problema de utilizar un evaluador externo propio es la fiabilidad, ya que no es posible conocer de antemano como se va a comportar cada modelo ante la tarea de evaluación propuesta. Sobre todo teniendo en cuenta que esta tarea requiere de un razonamiento complejo para saber interpretar si una respuesta se apoya en un contexto dado o si ese contexto es realmente suficiente para responder a la pregunta planteada.

Por esto es necesario conocer como se va a comportar el modelo antes de su uso. Con este objetivo se realizó una evaluación manual de una muestra de 100 preguntas seleccionadas del conjunto de evaluación. Para cada una de estas preguntas, junto con la respuesta generada por el sistema y el contexto que se le había introducido, se realizó un etiquetado manual de cada una de las cuatro métricas descritas anteriormente. La evaluación manual se consideró una referencia de base y se utilizó como *ground truth* para validar los distintos mecanismos automáticos de evaluación.

Con las métricas de referencia definidas se pasa a la elección del evaluador externo. La evaluación automática se plantea como una tarea de clasificación binaria para cada una de las métricas. Para ello se proporcionan al modelo evaluador la pregunta original, el contexto recuperado, la respuesta generada y la respuesta de referencia del dataset. A partir de esta información el modelo debe determinar si la información necesaria se encuentra presente en el contexto, si la respuesta contiene alucinaciones o si se ha producido una evasión incorrecta.

Se realizaron pruebas utilizando distintos modelos ejecutados localmente. Sin embargo, los resultados obtenidos mostraron una elevada variabilidad y una baja tasa de aciertos respecto a las etiquetas generadas manualmente. En particular, la detección de alucinaciones y la identificación de evasiones incorrectas resultaron especialmente problemáticas, produciendo clasificaciones inconsistentes entre ejecuciones y entre modelos diferentes, dando tasas de acierto entre el 30 % y el 60 %. Además, utilizar el mismo modelo o modelos de capacidades similares tanto para generar como para evaluar las respuestas introduce riesgos importantes, ya que los errores y sesgos presentes durante la generación pueden reproducirse también durante la evaluación.

Por este motivo, y ya que los requerimientos para calcular estas métricas requieren de un razonamiento superior que el proporcionado por los modelos pequeños y medios, se decidió utilizar un modelo de mayor capacidad exclusivamente para las tareas de evaluación. Dado que esta fase no forma parte de la arquitectura final del sistema y únicamente se emplea para el análisis y evaluación, se decidió recurrir a un servicio externo. La plataforma seleccionada fue Groq [22], que permite el uso de varios modelos potentes de gran tamaño y acceder gratuitamente, con unos límites de uso por tokens tanto por minuto como diarios. Esto lo hacía perfecto para poder hacer pruebas de rendimiento y ver la fiabilidad de los modelos ofrecidos.

En esta plataforma se ofrece el uso de diferentes modelos con distintas capacidades. Para este caso de uso se probaron 3 de los modelos que ofrecen mejor capacidad de razonamiento. dos de estos son modelos de OpenAI, `openai/gpt-oss-120b` y `openai/gpt-oss-20b` con 120 y 20 mil millones de parámetros respectivamente, el otro fue un modelo de la familia Llama de 70 mil millones de parámetros el `llama-3.3-70b-versatile`, elegido para la prueba ya que uno de esta familia de menor tamaño ha sido utilizado para las pruebas en local del sistema. A continuación se muestra la tasa de acierto de cada modelo para cada una de las métricas definidas en comparación con la muestra de referencia etiquetada manualmente:

Modelo	Answer Correctness	Hallucination	Context Recall	Incorrect Evasion
<code>openai/gpt-oss-120b</code>	98 %	97 %	98 %	98 %
<code>openai/gpt-oss-20b</code>	98 %	93.9 %	91.9 %	98 %
<code>llama-3.3-70b</code>	93.75 %	81.6 %	81 %	62.5 %

Tabla 3.4: Tasa de aciertos de los modelos como LLM-as-judge

En las pruebas realizadas se aprecia claramente como el modelo de 120 mil millones de parámetros sobrepasa a los otros dos, especialmente al de la familia de Llama, obteniendo más de un 97 % de aciertos en las cuatro métricas evaluadas. Por el contrario vemos que el modelo `llama-3.3-70b-versatile` se queda tan solo en un 81 % en la medida de *hallucination* y *context_recall* y que incluso cae hasta un 62 % en la evaluación de *incorrect_evasion*. Por estas razones se decide finalmente utilizar el modelo `openai/gpt-oss-120b` como *LLM-as-a-Judge* ya es el que aporta una mayor fiabilidad.

Con la selección del modelo evaluador queda completada la presentación de la arquitectura. En el capítulo siguiente se presentan los resultados obtenidos por esta arquitectura sobre los distintos experimentos realizados.

CAPÍTULO 4

Resultados

En este capítulo se presentan los resultados obtenidos tras las pruebas realizadas con la arquitectura así como las limitaciones tanto por el hardware utilizado como por el diseño de la arquitectura.

4.1 Hardware utilizado y limitaciones

Como se ha especificado a lo largo de este documento, todos los experimentos realizados en este trabajo se han ejecutado en un ordenador portátil personal, sin utilizar infraestructura especializada ni recursos externos para las tareas principales de extracción y construcción del grafo, la recuperación de la información y la generación de respuestas.

Las especificaciones del equipo utilizado son las siguientes:

- Procesador AMD Ryzen 7 5800H a 3.20 GHz.
- 16 GB de memoria RAM.
- Tarjeta gráfica NVIDIA GeForce RTX 3050 (4 GB).
- Sistema operativo Windows 11.

Uno de los propósitos de este trabajo es comprobar la viabilidad de construir sistemas híbridos RAG-KAG en entornos locales sin necesidad de recurrir a servicios externos o equipos con recursos computacionales avanzados.

La principal limitación del equipo utilizado ha sido sin duda la memoria RAM disponible. Se ha visto como una memoria de 16 GB es insuficiente para muchos procesos que involucran el uso de varias herramientas conjuntas, especialmente con modelos de lenguaje. Además, aunque el sistema dispone de 16 GB, la memoria realmente disponible es mucho menor debido al consumo del propio sistema operativo, la base de datos Neo4j, y el entorno de desarrollo debido al propio proceso de ejecución. Por esto, en la práctica, la memoria real disponible se sitúa entre los 6 y 9 GB bajo condiciones ideales, esto es, sin hacer uso de otros programas o procesos externos a la arquitectura.

La tarjeta gráfica disponible, una NVIDIA RTX 3050, permite acelerar determinadas tareas de procesamiento. Sin embargo, la cantidad de memoria de VRAM disponible limita también el tamaño de los modelos que pueden ejecutarse íntegramente en GPU, lo que no permite el uso de modelos de tamaño medio-grande.

Esta limitación ha condicionado directamente la selección de modelos de lenguaje, así como otras herramientas como modelos de *embeddings* o modelos de *reranking* utilizados durante el desarrollo. Muchos de los modelos requieren mucha más memoria de la disponible, incluso algunos considerados livianos y recomendados para entornos locales.

Las restricciones de memoria también afectaron al número de registros utilizados para las pruebas. Aunque el dataset 2WikiMultihopQA contiene más de 160.000 ejemplos para entrenamiento, trabajar con la totalidad del conjunto de datos supone tiempos de procesamiento y requisitos de almacenamiento incompatibles con los recursos disponibles. Por este motivo se ha optado por trabajar con subconjuntos lo suficientemente representativos para trabajar con tiempos de ejecución asequibles sin comprometer la validez de los resultados.

Aún con estas limitaciones se ha desarrollado una arquitectura completamente funcional, capaz de realizar todas las fases del proceso en tiempos razonables. Los resultados obtenidos demuestran la viabilidad de integrar este tipo de sistemas en equipos personales, aunque con ciertas limitaciones.

Respecto al evaluador externo, no fue posible realizar todas las consultas únicamente con el plan gratuito. Debido al elevado número de tokens consumidos se optó por la opción de pago. En este aspecto, Groq ofrece unos costes asumibles para el trabajo realizado. En total, para un consumo de aproximadamente 9 millones de tokens el coste ha sido de 1.8 dólares, incluyendo los resultados finales más una serie de pruebas realizadas, lo que confirma que el uso de un evaluador externo únicamente durante la fase de evaluación supone un coste reducido.

4.2 Resultados grafo de referencia

Para poder evaluar la arquitectura completa de una manera fiable es necesario partir de una base. Con este objetivo se utilizan los grafos de referencia, creados a partir de las evidencias aportadas por el dataset 2WikiMultihopQA así como de la unión de estas evidencias más las tripletas extraídas con REBEL. Estas evidencias son las entidades y relaciones mínimas para poder responder a cada una de las preguntas, lo que elimina la mayor parte del ruido que encontramos en un grafo construido a partir de la extracción automática.

Con el grafo creado a partir de las evidencias es posible determinar que capacidad tiene el sistema para contestar a las preguntas en con una extracción ideal, por lo tanto se puede comparar, en parte, con el escenario real e identificar que fallos vienen dados por el *retrieval* y cuales por la extracción.

Por otro lado, con el grafo creado a partir de la unión de las evidencias y de las tripletas extraídas automáticamente es posible aislar el rendimiento del proceso de recuperación del proceso de extracción. Esto se debe a que en el grafo real se introduce mucho ruido, por lo que es más difícil para el *retrieval* extraer la

información necesaria. Teniendo un grafo mixto, es posible conocer el rendimiento en el caso de saber que se tiene toda la información disponible con el ruido del resto de tripletas agregado. De esta manera, cuando se comparen estos resultados los obtenidos del grafo real, se podrá decidir que pérdida de rendimiento se debe exclusivamente a los fallos u omisiones en la extracción.

A continuación se puede ver una tabla con los resultados de las métricas propias del dataset, en esta se muestra la media de las métricas obtenidas para el total de las preguntas. Para las pruebas de los grafos de referencia se ha hecho uso del modelo de lenguaje qwen2.5-7b:

Grafo	em_mean	precision_mean	recall_mean	f1_mean
Grafo_referencia (solo evidencias)	0.8	0.79	0.8	0.79
Grafo_referencia (evidencias + REBEL)	0.71	0.73	0.74	0.73

Tabla 4.1: Resultados métricas dataset grafo de referencia

A partir de estos resultados ya se pueden extraer algunas conclusiones. Con el grafo construido únicamente con las evidencias, el sistema es capaz de contestar correctamente el 80 % de las preguntas, obteniendo resultados muy similares en las 4 métricas. Con el añadido de la información extraída con REBEL, el número de respuestas que coinciden exactamente con la referencia cae 9 puntos hasta el 71 %. En los valores de precisión, recall y F1 esta caída es algo más ligera, de 6 puntos. Estos datos sugieren que la introducción de la información adicional obtenida mediante extracción automática influye directamente en el módulo de recuperación de información pero moderadamente.

Sin embargo, como se comentó en la sección de métricas, las proporcionadas por el dataset solo evalúan por comparación directa de tokens, haciendo que respuestas correctas queden evaluadas como incorrectas o viceversa. Por esta razón se muestran también las 4 métricas obtenidas mediante el *LLM-as-a-Judge*, para corroborar el buen funcionamiento del *retrieval* y descartar alucinaciones. Al igual que en la tabla anterior se muestra la media de las puntuaciones para cada una de las métricas:

Grafo	answer correctness	hallucination	context recall	incorrect evasion
Grafo_referencia (solo evidencias)	0.8	0.04	0.89	0.07
Grafo_referencia (evidencias + REBEL)	0.74	0.08	0.87	0.11

Tabla 4.2: Resultados métricas propias grafos de referencia

A partir de estas métricas podemos obtener algunas conclusiones adicionales. Lo primero que resalta es el valor del *context_recall*, de un 0.89 y 0.87 para ambos grafos. Esto indica que en un 89 % y un 87 % respectivamente de los casos analizados, sí se recuperó el contexto correcto y el modelo tenía la información necesaria para responder, sin embargo los valores de respuesta correcta caen hasta el 0.8 y

0.74 respectivamente. Esto supone que el módulo de recuperación funciona mejor de lo que se deduce de las métricas oficiales, ya que, bajo condiciones ideales, es capaz de recuperar la información necesaria en un 89 % de los casos, 9 puntos más de lo que sugerían las métricas oficiales. Este valor es coherente con el resto de métricas, ya que se observa que el número de evasiones incorrectas en el caso ideal es del 7 %, es decir, casos en los que sí se disponía la información necesaria pero el sistema no supo interpretarla y contestó con una evasiva.

Por otro lado se deduce que la pérdida de calidad al añadir al grafo ideal la información extraída por el módulo de extracción automática no es tan acentuada como indicaban las métricas anteriores, sino que se debe en mayor medida a la respuesta generada por el LLM. Esto se sustenta en que el *context_recall* tan solo se reduce en un 2 % al introducir esta nueva información, mientras que el número de respuestas correctas cae un 6 % y el número de evasiones incorrectas se incrementa un 4 %. Estos datos sustentan la afirmación de que el módulo de *retrieval* es capaz de mantener su rendimiento prácticamente constante a pesar de aumentar la cantidad de información en la base de datos hasta 8 veces más, sin embargo el LLM sufre cuando la información que le llega es más de la necesaria para contestar la pregunta.

Esta caída en el rendimiento, como se comprueba, se debe al LLM, aunque también se puede hacer la lectura de que el módulo de *retrieval* tiene margen de mejora a la hora de filtrar el contexto final, ya que con esto se demuestra que el LLM es sensible a la introducción de información irrelevante para la tarea en cuestión, y por tanto, la única tarea del *retrieval* no debe limitarse únicamente a proporcionar la información necesaria sino también filtrarla lo máximo posible para reducir al máximo la introducción de ruido.

4.3 Resultados grafo generado con REBEL

Una vez conocido el rendimiento del sistema bajo condiciones ideales mediante el grafo de referencia se procede a analizar el sistema con datos reales, con el grafo construido a partir de la extracción automática mediante REBEL.

Al contrario que en el grafo de referencia, este contiene errores derivados de la extracción y es posible que la información necesaria para contestar alguna de las preguntas no se haya incluido. Entre los posibles errores están la duplicación de entidades que hayan pasado el filtro de deduplicación o por el contrario, nodos que se hayan unido sin ser la misma entidad por falsos positivos, entidades faltantes, relaciones incorrectas, duplicación de información, redundancia, etc.

El objetivo principal por tanto es el de analizar el impacto de la extracción automática y el del comportamiento de diferentes modelos de lenguaje bajo las limitaciones técnicas comentadas anteriormente.

4.3.1. Resultados globales

Lo primero es comparar estos resultados con los obtenidos con los grafos de referencia, para poder conocer la diferencia entre el comportamiento bajo unos

parámetros ideales y bajo unos reales y comprobar el nivel de variación en la calidad final de las respuestas. Para esta comparativa se ha utilizado el mismo modelo de lenguaje, el qwen2.5-7b, así como los mismos parámetros y condiciones del *retrieval* para obtener una comparación lo más precisa posible.

Grafo	Answer Correctness	Aallucination	Context Recall	Incorrect Evasion
Grafo referencia ideal (solo evidencias)	0.8	0.04	0.89	0.07
Grafo referencia con ruido (evidencias + REBEL)	0.74	0.08	0.87	0.11
Grafo extraído (solo REBEL)	0.5	0.199	0.515	0.073

Tabla 4.3: Comparativa grafos de referencia con grafo real

Esta tabla permite cuantificar la caída en el rendimiento debido a utilizar únicamente los datos de la extracción automática. Es especialmente relevante el dato del context recall, que cae de un 87 % a tan solo un 51.5 %. Esto indica que la extracción de REBEL dista mucho de ser perfecta y omite mucha información relevante, ya que los casos donde se consigue recuperar la información necesaria caen en más de un 35 %, lo que se debe exclusivamente al proceso de extracción. Esto, lógicamente, tiene un impacto directo en las respuestas correctas aportadas por el sistema, que caen en la misma proporción hasta un 50 %, lo que supone que en la mitad de las preguntas no se ha conseguido encontrar la respuesta correcta. También puede observarse la tendencia a aumentar las alucinaciones cuando el modelo no tiene el contexto necesario, lo que indica que ante la ausencia de este contexto, el modelo tiende a responder y por tanto a alucinar. Por otro lado, puede sorprender que la tasa de evasiones incorrectas descienda respecto al grafo de referencia que contenía tanto las tripletas aportadas por el dataset como las extraídas. Pero esto se debe a una cuestión estadística, en el nuevo grafo hay un 36 % menos de los casos donde se ofrece la información necesaria al LLM, por lo tanto hay muchos más casos donde una evasiva es correcta, lo que deriva en una reducción de la probabilidad de una evasión incorrecta.

Con esta comparativa ya se conoce la medida en que la caída en la tasa de recuperación del contexto correcto, y por tanto del número de respuestas correctas, se debe a las imprecisiones en el proceso de extracción y construcción del grafo. A partir de este punto se analizan únicamente los resultados obtenidos a partir del grafo de estudio, obtenido mediante la extracción automática no supervisada de información.

Para el estudio de la influencia y rendimiento del modelo de lenguaje en la generación de respuestas se evalúan 7 modelos distintos ejecutados localmente. Todos ellos se ejecutan bajo la misma configuración de *retrieval*, contexto recuperado y prompt, donde únicamente cambia el modelo utilizado. De esta forma es posible conocer el impacto de los distintos modelos en la arquitectura final.

La elección de modelos se hace de tal forma que existan diferentes tamaños con diferentes capacidades para comprobar el límite del sistema y los rendimientos. Los modelos utilizados son todos de tamaños medio y pequeños, ya que todos aquellos que utilizan más de 10 mil millones de parámetros exceden las capaci-

dades en términos de memoria RAM del hardware utilizado por lo que no han podido ser probados.

Los modelos evaluados han sido Gemma3-4B (4 mil millones de parámetros), Gemma-7B (7 mil millones), Qwen2.5-3B (3 mil millones), qwen2.5-7b (7 mil millones), Llama3-8B (8 mil millones), Mistral-7B (7 mil millones) y Phi3 (3.8 mil millones). Todos ellos se ejecutaron utilizando la misma configuración de *retrieval* y *prompting*, de forma que las diferencias observadas en los resultados puedan atribuirse únicamente al comportamiento del modelo durante la generación de respuestas.

Otros modelos como phi4 y DeepSeek-R1:14b con 14 mil millones de parámetros podían llegar a funcionar, pero a costa de introducir una enorme latencia en las respuestas y de experimentar caídas en el sistema por lo que se descartó su uso.

En los resultados se puede encontrar tanto la evaluación a partir de las métricas oficiales del dataset como a partir de las métricas propias. Las primeras permiten comparar los resultados con otros trabajos que hacen uso de este dataset mientras que las métricas propias permiten analizar de una forma independiente la calidad de recuperación de información del rendimiento del modelo de lenguaje.

Primero se muestran las métricas oficiales del dataset obtenidas con cada uno de los modelos:

Modelo	EM	Precision	Recall	F1
Gemma3-4B	0.512	0.533	0.551	0.539
Qwen2.5-7B	0.479	0.491	0.502	0.494
Llama3-8B	0.442	0.470	0.513	0.479
Qwen2.5-3B	0.426	0.446	0.457	0.450
Mistral-7B	0.415	0.446	0.497	0.456
Phi3-mini	0.296	0.330	0.379	0.340
Gemma-7B	0.240	0.268	0.378	0.283

Tabla 4.4: Resultados obtenidos mediante las métricas oficiales de 2WikiMultihopQA

De estos resultados podemos ya ver el buen rendimiento que ofrece Gemma3-4B, que obtiene las mejores puntuaciones en todos los aspectos. Sorprendentemente supera a modelos de mayor tamaño, especialmente a su predecesor Gemma-7B, que con el doble de parámetros obtiene puntuaciones un 50 % más bajas. Otro modelo que obtiene métricas muy bajas es Phi3, pero como se verá a continuación esto se debe a la propia evaluación mediante el uso de métricas basadas en coincidencia textual, lo que no mide con precisión la calidad real de las respuestas.

A continuación se muestran las métricas adicionales obtenidas mediante la técnica de *LLM-as-a-Judge*.

Modelo	Answer Corr.	Context Recall	Hallucination	Inc. Evasion
Gemma3-4B	0.544	0.516	0.318	0.016
Llama3-8B	0.518	0.512	0.199	0.044
Qwen2.5-7B	0.503	0.515	0.145	0.073
Phi3	0.502	0.514	0.255	0.065
Mistral-7B	0.473	0.515	0.208	0.064
Qwen2.5-3B	0.458	0.508	0.199	0.066
Gemma-7B	0.370	0.518	0.150	0.100

Tabla 4.5: Resultados obtenidos mediante las métricas adicionales propuestas

Los resultados obtenidos mediante las métricas adicionales permiten analizar aspectos del sistema que no quedan reflejados en las métricas oficiales del dataset. Las variaciones en el *context_recall* se deben exclusivamente a la evaluación del modelo `openai/gpt-oss-120b` que hace de juez, ya que el contexto proporcionado en todos los casos es el mismo, y por tanto esta métrica siempre debería obtener el mismo valor. Sin embargo, el modelo varía ligeramente su evaluación en las diferentes ejecuciones, aunque como se observa, esta variación es de máximo un 1 %, que se encuentra dentro de los límites aceptados.

En cuanto a la calidad de las respuestas, Gemma3-4B vuelve a obtener el mejor resultado en la métrica *Answer Correctness*, alcanzando un valor del 54.4 % de respuestas correctas. Este resultado confirma las observaciones realizadas con las métricas oficiales y sitúa a este modelo como el más efectivo de los evaluados para este escenario. Sin embargo, este mejor rendimiento va acompañado de una mayor tasa de alucinaciones, alcanzando un valor de 31.8 %, significativamente superior al resto de modelos. Esto indica que el modelo tiende a responder con mayor frecuencia incluso cuando el contexto recuperado no contiene la información necesaria para justificar la respuesta, lo que compromete la fiabilidad del sistema.

Por el contrario, modelos como `qwen2.5-7b` o `Gemma-7B` presentan tasas de alucinación considerablemente menores, aunque también obtienen peores resultados en las métricas de calidad de respuesta. Esto implica un compromiso entre precisión y prudencia donde algunos modelos optan por responder un mayor número de veces aunque no tengan la información, asumiendo mayor riesgo. Otros, por el contrario, son más conservadores y tienden a alucinar menos, lo que implica un menor número de aciertos.

Aquí se comprueba lo comentado anteriormente sobre Phi3. Aunque sus resultados en EM y F1 se encuentran entre los más bajos de todos los modelos analizados, consigue una puntuación de *Answer Correctness* cercana a la de modelos superiores según las métricas oficiales. Esto hace ver la fragilidad de las métricas oficiales, ya que respuestas conceptualmente correctas pueden recibir una puntuación muy baja si no coinciden exactamente con el formato esperado por el dataset. Esto también indica que este modelo tiene la capacidad de interpretar bien el contexto aunque tiene un rendimiento inferior a la hora de seguir instrucciones estrictas, como en este caso, dar respuestas cortas y concisas.

Finalmente, la métrica *Incorrect Evasion* muestra diferencias importantes entre modelos. Gemma3-4B destaca por presentar la menor tasa de evasiones incorrectas, lo que significa que es capaz de aprovechar mejor la información recuperada cuando se le proporciona. Al contrario que lo que ocurre con Gemma-7B, que obtiene el valor más elevado, lo que sugiere una mayor tendencia a responder con

I don't know incluso en situaciones donde el contexto proporcionado contiene información suficiente para generar una respuesta correcta.

A continuación se muestran las métricas derivadas de las anteriores, lo que permite conocer con más detalle el funcionamiento del sistema:

Tabla 4.6: Métricas derivadas para el análisis del comportamiento del sistema

Modelo	Correctness Supported	Correctness Hallucination	Missed Retrieval
Gemma3-4B	0.422	0.120	0.094
Llama3-8B	0.428	0.086	0.084
Qwen2.5-7B	0.414	0.091	0.101
Phi3	0.403	0.095	0.111
Mistral-7B	0.401	0.070	0.114
Qwen2.5-3B	0.367	0.092	0.141
Gemma-7B	0.356	0.016	0.162

La métrica *Correctness Supported* indica los casos en los que la respuesta generada es correcta y además se basa en el contexto recuperado. Esta representa el funcionamiento integral de la arquitectura, ya que requiere que las fases de extracción, recuperación y generación funcionen correctamente. En este caso se impone Llama3 con un 42.8 %, seguido muy de cerca por Gemma3-4B con 42.2 % y qwen2.5-7b con 41.4 %. La escasa diferencia entre estos modelos indica que, una vez que la información correcta ha sido recuperada, la mayoría son capaces de utilizarla adecuadamente para construir la respuesta.

La métrica *Correctness Hallucination* identifica las respuestas consideradas correctas a pesar de no existir información suficiente dentro del contexto recuperado, por lo que es un parámetro clave para conocer la fiabilidad. Debido a que muchas respuestas son únicamente de si/no y se fuerza al modelo mediante el prompt a dar respuestas cortas, es habitual que esto ocurra. Gemma3-4B es el modelo que más sufre este efecto, con hasta un 14 % de los casos, seguido de Phi3 y Qwen2.5-3B. Este resultado coincide con la elevada tasa de alucinaciones observada anteriormente y de él se puede deducir que parte del buen rendimiento de Gemma3-4B proviene de respuestas que no están completamente justificadas por la información proporcionada por el sistema de recuperación.

Por otro lado, la métrica *missed retrieval* permite identificar situaciones en las que el contexto recuperado contiene información suficiente para responder correctamente pero el modelo no es capaz de aprovecharla. Aquí, Llama3 vuelve a obtener el mejor resultado con un valor inferior al 10 %, mientras que Gemma-7B ofrece el peor resultado, ya que presenta esta limitación en un 16 % de los casos. Este comportamiento da a entender que algunas de las diferencias observadas entre modelos no se encuentran en la recuperación de información sino en la capacidad para interpretar correctamente el contexto disponible, lo que refuerza la las observaciones analizadas en la comparación de grafos de referencia.

En general, estas métricas muestran que las diferencias entre modelos no pueden explicarse únicamente mediante las métricas tradicionales de evaluación. Mientras algunos modelos obtienen mejores resultados debido a una mayor capacidad para interpretar el contexto recuperado, otros obtienen puntuaciones elevadas basando sus respuestas en conocimiento interno no respaldado por la información proporcionada por el sistema. Esta distinción resulta especialmente

útil en la evaluación de arquitecturas RAG-KAG, donde uno de los objetivos principales es que las respuestas estén basadas en información recuperada y no únicamente en el conocimiento previo del modelo o en alucinaciones.

4.3.2. Resultados por tipo de pregunta

Las métricas vistas hasta ahora permiten obtener una visión general del rendimiento de la arquitectura, pero todavía es posible analizar la calidad del sistema en función del razonamiento requerido para contestar los distintos tipos de preguntas del dataset. En esta sección se analizan las diferencias existentes entre los distintos tipos de preguntas de 2WikiMultihopQA, que incorpora diversas categorías con características y niveles de complejidad diferentes.

Este análisis permite comprobar con que tipos de razonamiento funciona mejor el sistema y cuáles le suponen un mayor desafío. Además, permite ver si los diferentes modelos de lenguaje tienen un mejor o peor comportamiento a la hora de contestar algún tipo en concreto.

A continuación se presentan los resultados de F1 agrupados según las categorías definidas en el dataset: *composition*, *comparison*, y *bridge comparison*.

Modelo	Bridge	Comparison	Compositional
Gemma3-4B	0.663	0.574	0.479
Qwen2.5-7B	0.667	0.567	0.389
Llama3	0.556	0.492	0.446
Mistral	0.399	0.516	0.447
Phi3	0.108	0.362	0.428
Qwen2.5-3B	0.691	0.391	0.404
Gemma-7B	0.189	0.255	0.351

Tabla 4.7: F1 por tipo de pregunta

Los resultados muestran diferencias importantes entre los distintos tipos de preguntas. La categoría que obtiene mejores resultados es *bridge comparison*. En esta categoría sobresalen especialmente Qwen2.5-3B, qwen2.5-7b y Gemma3-4B, todos ellos con valores de F1 superiores al 0.65.

Las preguntas de tipo *comparison* presentan una dificultad intermedia. Aunque también obtienen resultados elevados para varios modelos, se observa una reducción generalizada respecto a *bridge comparison*. Esto indica que la recuperación de la información necesaria sigue siendo efectiva, pero la fase de generación requiere una mayor capacidad para interpretar correctamente los datos recuperados.

Por último, las preguntas de tipo *compositional* aparecen como las más exigentes a la hora de ser contestadas en la mayoría de los casos. Estas respuestas requieren conectar varios nodos mediante más de una relación y comparar información proveniente de caminos distintos. Esto provoca una dificultad añadida tanto para el *retrieval*, que tiene que encontrar los caminos independientes, como para el modelo de lenguaje, que debe interpretar correctamente una comparación entre entidades no relacionadas. Aunque los resultados siguen siendo razonables, se observa una caída de rendimiento en prácticamente todos los modelos. Esto hace pensar que las limitaciones de la arquitectura se acentúan en este tipo

de preguntas en las que tanto la recuperación del contexto como su posterior interpretación aumentan la complejidad de la tarea.

En términos generales, se confirma que el sistema desarrollado es más efectivo a la hora de contestar preguntas que requieren recuperar información de nodos vecinos conectados en el grafo, pero por otro lado sufre más cuando estas preguntas requieren conectar información procedente de caminos independientes y no conectados entre sí.

Ahora se analiza el *context recall* por tipo de pregunta. Con esta métrica se puede comprobar si el *retrieval* tiene realmente más dificultad para recuperar algún tipo de pregunta respecto a otra.

Modelo	Bridge	Comparison	Compositional
Gemma3-4B	0.500	0.503	0.539
Qwen2.5-7B	0.490	0.509	0.537
Llama3	0.505	0.494	0.535
Mistral	0.510	0.497	0.537
Phi3	0.526	0.500	0.526
Qwen2.5-3B	0.485	0.491	0.537
Gemma-7B	0.510	0.512	0.532

Tabla 4.8: Context Recall por tipo de pregunta

Se observa que los resultados de context recall apenas varían entre los tipos de pregunta. En todos los casos los valores se sitúan entre 48 % y 54 %, lo que indica que el tipo de pregunta apenas influye en la calidad del *retrieval* diseñado.

Con esto también se deduce que las diferencias en la tasa de respuestas correctas en función del tipo de pregunta se deben casi exclusivamente al modelo utilizado y no a la recuperación de la información.

A continuación se analiza el *correctness supported*, casos donde la respuesta correcta está respaldada completamente por el contexto aportado.

Modelo	Bridge	Comparison	Compositional
Gemma3-4B	0.388	0.379	0.487
Qwen2.5-7B	0.417	0.440	0.410
Llama3	0.381	0.425	0.457
Mistral	0.296	0.385	0.468
Phi3	0.326	0.367	0.470
Qwen2.5-3B	0.402	0.298	0.415
Gemma-7B	0.276	0.331	0.424

Tabla 4.9: Correctness Supported por tipo de pregunta

Los resultados muestran generalmente valores más elevados para las preguntas de tipo composición. Esto contrasta con la métrica de F1, donde se deducía que estas preguntas eran más difíciles de contestar correctamente para la arquitectura. Con esto se demuestra que si el contexto recuperado es correcto, los modelos son capaces de utilizarlo bien en muchas ocasiones, y que en las preguntas de tipo *bridge comparison* se suelen producir más alucinaciones, lo que explica mejores valores en f1 que caen varios puntos cuando se comprueba si realmente están respaldados por el contexto.

Los modelos qwen2.5-7b y Llama3 presentan los resultados más equilibrados entre las distintas categorías, mientras que Mistral y Phi3 son menos consistentes y reflejan diferencias de hasta el 15 % entre las diferentes tipologías.

En general, estos resultados hacen ver que la arquitectura es capaz de proporcionar la información correcta independientemente del tipo de pregunta, y que el resultado final depende en mayor medida del modelo utilizado.

Por último se muestra la métrica *missed retrieval*. Con esta se puede ver con claridad los casos en los que realmente la responsabilidad de una respuesta incorrecta es del modelo, al tener un contexto adecuado para responder la pregunta pero no ser capaz de utilizarlo.

Modelo	Bridge	Comparison	Compositional
Gemma3-4B	0.112	0.124	0.052
Qwen2.5-7B	0.073	0.069	0.127
Llama3	0.124	0.069	0.078
Mistral	0.214	0.112	0.069
Phi3	0.200	0.133	0.057
Qwen2.5-3B	0.082	0.193	0.122
Gemma-7B	0.235	0.181	0.108

Tabla 4.10: Missed Retrieval por tipo de pregunta

En este caso se aprecian claras diferencias entre los diferentes modelos. Al igual que en resultados anteriores, los peores resultados corresponden a Gemma-7B, Mistral y Phi3, que muestran tasas de desaprovechamiento especialmente malas en preguntas tipo bridge, donde no supieron aprovechar el contexto en más del 20 % de los casos.

Por otro lado, sobre el resto de modelos, destaca que los de la familia Qwen2.5, tanto en su versión de 7 mil millones como en la de 3 mil millones de parámetros consiguen rendimientos considerablemente mejores al resto en preguntas tipo bridge.

Un resultado relevante es que las preguntas de tipo compositional no presentan los mayores valores de *missed retrieval*. De hecho, para algunos de los modelos representan la categoría donde esta métrica obtiene sus mínimos. Lo contrario ocurre con las preguntas de tipo bridge-comparison, donde en general todos los modelos obtienen la tasa más alta de omisión del contexto, con casos como Mistral, Phi3 o Gemma-7B donde ocurre en más del 20 % de los casos.

Este comportamiento se complementa con el observado con anterioridad. La recuperación correcta de la información por el *retrieval* es similar independientemente del tipo de pregunta, sin embargo, las respuestas correctas son menores en las preguntas tipo bridge. Esto se debe directamente a la influencia del modelo, como se corrobora con esta última métrica, donde se comprueba que en este tipo de preguntas algunos modelos tienden a no aprovechar la información proporcionada aunque esta sea correcta.

4.3.3. Análisis de rendimiento y recursos

Con el objetivo de analizar el coste computacional asociado al sistema de recuperación, se registró el tiempo necesario para recuperar la información para

cada una de las 500 preguntas evaluadas. Esta medición se realiza tanto sobre los grafos de referencia como sobre el grafo generado automáticamente mediante REBEL. En la tabla se omiten los resultados del grafo de referencia que contiene las evidencias y las tripletas extraídas por REBEL debido a su similitud con el grafo final del trabajo, puesto que el número de nodos y relaciones varía únicamente en un 2 % en el número de nodos y relaciones

Métrica	Grafo ideal	Grafo REBEL
Tiempo medio (s)	0.54	20.64
Desviación estándar (s)	0.97	51.12
Mediana (s)	0.36	3.3
Tiempo máximo (s)	19.97	400.19

Tabla 4.11: Estadísticas de los tiempos de recuperación por pregunta

Los resultados muestran diferencias marcadas entre ambos escenarios. En el caso del grafo de referencia, el tiempo medio de recuperación es del 0.5 segundos por pregunta. Este es un valor esperado ya que al disponer únicamente de la información estrictamente necesaria el proceso de extracción y filtrado es prácticamente directo y el sistema no requiere invertir tiempo en este paso.

Por el contrario, en el escenario de extracción automática, se obtiene un tiempo medio de recuperación por pregunta de 20.64 segundos. Sin embargo, este valor no refleja la verdadera latencia de este paso, como se observa en la tabla, la desviación estándar es de más de 51 segundos y se observa un caso que incluso llegó a superar los 400 segundos en la recuperación de información. Por esto se deduce que esta media está influenciada por algunos casos extremos que distorsionan la medida total. Es por esto que un dato que representa mejor la latencia de esta fase es por tanto la mediana, que se sitúa en 3.3 segundos, lo que supone un valor más que aceptable para los recursos del dispositivo utilizado.

El origen de estos casos se debe a la presencia de supernodos con grados especialmente elevados dentro del grafo. Cuando uno de estos nodos es vecino de alguna de las entidades detectadas en la pregunta, se genera un número muy elevado de caminos posibles, por lo que las tareas de filtrado y *reranking* se demoran en exceso.

A pesar de estos casos extremos, la mayoría de los casos consiguen resolverse en tiempos aceptables, lo que indica que el principal problema de rendimiento se deriva de situaciones concretas asociadas a la topología del grafo.

El consumo de memoria RAM ha sido otro de los desafíos en el desarrollo de esta arquitectura. Durante las pruebas se comprobó que un gran número de modelos orientados a funcionar en entornos locales no pueden ejecutarse con 16 GB. Se ha visto que los modelos de lenguaje requieren unos mínimos antes de empezar a procesar aunque luego no hagan uso de ese requerimiento en su totalidad. A continuación se muestran los valores de memoria consumida por todo el proceso en función de los modelos utilizados durante las pruebas. Cabe destacar que este es el uso de memoria únicamente por el proceso de python, el consumo de otros componentes como la base de datos Neo4j no están reflejados en estos valores.

Tras las pruebas se observan valores de utilización de memoria muy similares, con pocas variaciones incluso en modelos con una diferencia de más del doble

Modelo	Consumo máximo de RAM (GB)
Gemma3-4B	3.48
Qwen2.5-7B	3.58
Llama3	2.43
Mistral	3.08
Phi3	2.96
Qwen2.5-3B	3.08
Gemma-7B	2.9

Tabla 4.12: Comparativa de consumos RAM

en el número de parámetros utilizados. Esto se debe principalmente a la tarea requerida, ya que los prompts utilizados se componen de un número de tokens ajustado y las respuestas de salida se ajustan para que sean lo más cortas posibles y cumplir así con el estándar del dataset.

Estos resultados sugieren que para tareas simples de pregunta-respuesta es preferible utilizar siempre el modelo que mejor rendimiento dé en las pruebas, independientemente de su tamaño, siempre que se pueda ejecutar en el entorno de trabajo.

CAPÍTULO 5

Conclusiones

Los resultados obtenidos durante la evaluación demuestran que la construcción de una arquitectura híbrida basada en grafos de conocimiento es viable en entornos locales pero que tiene ciertas limitaciones, como el uso de modelos con altas capacidades de razonamiento o una baja latencia en algunos casos.

Respecto a la arquitectura, queda claro que una de las partes más exigentes y que aporta mayores desafíos es la de extracción de la información y construcción del grafo. Especialmente en entornos sin un esquema predefinido y sin conocer de antemano la información a extraer.

Para futuros trabajos es preferible trabajar con conjuntos de datos conocidos, y con un objetivo de uso predefinido. Esto permitirá conocer los requerimientos de la extracción, poder definir un esquema de nodos y relaciones a extraer, y así obtener toda la información necesaria para satisfacer las demandas del sistema.

Otra de las principales conclusiones es que las métricas tradicionales utilizadas en tareas de pregunta-respuesta no son suficientes para evaluar correctamente arquitecturas RAG-KAG. Durante la revisión de los resultados se observaron varios casos en los que respuestas consideradas incorrectas por las métricas clásicas eran realmente válidas, pero no se daban por buenas por estar compuestas por diferentes tokens. Y por el contrario, se han detectado varios casos donde las que respuestas correctas se obtenían sin estar respaldadas por la información recuperada. Esto evidencia que métricas como Exact Match o F1 son útiles para comparar sistemas, pero esconden el comportamiento interno real del sistema. Por esto, el uso de métricas orientadas a evaluar la calidad del *retrieval* y de aspectos como las alucinaciones es fundamental.

Con los resultados obtenidos se puede observar que la calidad de la respuesta final no depende únicamente de la capacidad del modelo de lenguaje. Se ha comprobado que las fases de *retrieval* y generación introducen distintos desafíos y deben evaluarse independientemente. Mientras que las métricas relacionadas con la recuperación de información son muy estables entre modelos, las diferencias en la calidad final de las respuestas fueron marcadas. Esto indica que gran parte de los errores observados se producen durante la interpretación del contexto recuperado y no durante la búsqueda de la información.

Otro punto importante a tener en cuenta es el compromiso que existe en los modelos de lenguaje entre capacidad de respuesta y fiabilidad. Los modelos con mejores tasas de acierto en las respuestas fueron también los que presentaron ma-

yores tasas de alucinación. Por el contrario, algunos modelos con comportamientos más conservadores ofrecieron menos respuestas incorrectas, aunque en algunos casos a costa de no contestar cuando sí disponían de información suficiente.

Los resultados muestran que no existe un modelo claramente superior en todos los aspectos y que la elección debe realizarse en función de las prioridades del sistema. En casos donde la fiabilidad es un requisito crítico puede ser preferible un modelo más conservador, mientras que en otros casos puede aceptarse una tasa mayor de alucinaciones para conseguir una tasa mayor de respuestas correctas.

5.1 Limitaciones

Con las conclusiones extraídas, es necesario señalar que este trabajo presenta una serie de limitaciones derivadas de las decisiones de diseño escogidas y del alcance del estudio realizado. Estas limitaciones impiden que el diseño de esta arquitectura pueda ser replicado en otros escenarios con el mismo rendimiento.

Una de las causas se debe al uso exclusivo del dataset 2WikiMultihopQA. A pesar de ser uno de los benchmarks de referencia en el área, y de contar con varios tipos de preguntas, todas ellas siguen un mismo patrón, por lo que no se puede conocer con precisión el comportamiento con preguntas de otra clase. Por otro lado, todos los textos proceden de artículos de Wikipedia, por lo que el rendimiento sobre datos de otra tipología, como documentos técnicos o dominios especializados, puede variar debido a la estructura gramatical y el vocabulario utilizado.

Sobre el sistema de recuperación, ha sido diseñado para trabajar sobre caminos de hasta dos saltos dentro del grafo. Esta decisión obedece a los requerimientos de las preguntas planteadas por el dataset y a un compromiso entre cobertura y ruido, pero limita la capacidad del sistema para contestar preguntas que requieran razonamientos con un mayor número de saltos entre entidades.

Algunos de los componentes del sistema han sido ajustados experimentalmente mediante el dataset utilizado, como los filtros de deduplicación o los umbrales de recuperación, filtrado *reranking*. Estos valores ofrecen un buen rendimiento para este caso de uso pero no se puede afirmar que vayan a ser óptimos para otros escenarios u otros grafos de conocimiento.

Otro punto a tener en cuenta es la evaluación de los resultados con el método de *LLM-as-a-Judge*. Aunque se comprobó una elevada tasa de aciertos respecto al etiquetado manual, las métricas adicionales propuestas siguen dependiendo de un LLM externo, por lo que los resultados no representan una verdad completamente objetiva.

Por último, el número de registros utilizados dentro del dataset es de 500, con lo que se extrajo un grafo de aproximadamente 13000 entidades. Aunque este tamaño es suficiente para evaluar el comportamiento del sistema, no se ha evaluado el funcionamiento sobre grafos de órdenes de magnitud superiores, donde los tiempos de recuperación y el consumo de memoria pueden variar significativamente, así como la calidad del *retrieval*.

5.2 Trabajo futuro

Tras desarrollar las conclusiones y analizar las limitaciones se detallan una serie de propuestas que pueden contribuir a mejorar la arquitectura. Estas mejoras, además de incrementar el rendimiento del sistema, también pueden ayudar al desarrollo de futuros trabajos en el ámbito de los sistemas RAG-KAG basados en grafos de conocimiento.

Como se ha señalado en varias ocasiones durante este trabajo, una de las partes más críticas se encuentra en la extracción automática de conocimiento y construcción del grafo, ya que condiciona el rendimiento y calidad general del sistema. En este aspecto, se propone el estudio de nuevos modelos de extracción de tripletas, e incluso adoptar estrategias híbridas donde se combinen los resultados de diferentes modelos especializados en esta tarea junto con extracciones a partir de LLMs para maximizar la cantidad y calidad de la información extraída.

Además de las estrategias de extracción, resulta interesante analizar el comportamiento del sistema sobre otros conjuntos de datos. Para futuros trabajos sería conveniente evaluar el uso de otros datasets especializados en razonamiento multi-salto, como HotpotQA o MuSiQue, así como sobre documentos de campos específicos. Esto permitiría una evaluación del comportamiento bajo distintas condiciones y conocer la capacidad de generalización de la arquitectura.

Otro aspecto de mejora consiste en optimizar el proceso de deduplicación de entidades. Actualmente, el método se basa en métricas de similitud textual. Este enfoque queda restringido por el riesgo de la detección de falsos positivos, lo que limita su capacidad para maximizar la desambiguación de verdaderos positivos. Futuros trabajos podrían incluir técnicas más avanzadas capaces de utilizar el contexto tanto en el grafo como en los textos de origen para mitigar este problema.

Respecto a la fase de recuperación, una evolución esperada del sistema consiste en implementar mecanismos de recuperación dinámica respecto al número de saltos. Evitando así restringirlo a búsquedas de vecinos de segundo orden y adaptando la profundidad en función de la complejidad y necesidades de la pregunta.

Con la implementación de estas mejoras sería posible conseguir arquitecturas más robustas, permitiendo su implementación en entornos reales y no solo de investigación. De esta manera, podría desarrollarse un sistema que integrase información de diversas fuentes permitiendo incorporar nuevo conocimiento de una forma robusta y escalable.

Bibliografía

- [1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
karpukhin2020dpr
- [2] Vladimir Karpukhin, Barlas Oguz, Sewon Min, et al. Dense Passage Retrieval for Open-Domain Question Answering. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 6769–6781, 2020.
- [3] Xiangrong Zhu, Yuexiang Xie, Yi Liu, Yaliang Li and Wei Hu. Knowledge Graph-Guided Retrieval Augmented Generation. *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL)*, 8912–8924, 2025.
- [4] Yue Shen, Yaliang Li, Nan Duan, et al. Retrieval-Augmented Generation with Knowledge Graphs: A Survey. *arXiv preprint arXiv:2408.08921*, 2024.
- [5] Yuan Feng, Jiahao Shi, Wei Wei, et al. Graph Retrieval-Augmented Generation: A Survey. *arXiv preprint arXiv:2501.00309*, 2025.
- [6] Bhaskarjit Sarmah, Dhagash Mehta, Benika Hall, Rohan Rao, Sunil Patel and Stefano Pasquali. HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction. *Proceedings of the 5th ACM International Conference on AI in Finance (ICAIF)*, 608–616, 2024.
- [7] Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh V. Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson and Bryan Hooi. G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering. *Advances in Neural Information Processing Systems (NeurIPS 2024)*, vol. 37, 2024.
- [8] Dongzhuoran Zhou, Yuqicheng Zhu, Yuan He, Jiaoyan Chen, Evgeny Kharlamov and Steffen Staab. Evaluating Knowledge Graph Based Retrieval Augmented Generation Methods under Knowledge Incompleteness. *arXiv preprint arXiv:2504.05163*, 2025.
- [9] Qiming Zeng, Xiao Yan, Hao Luo, Yuhao Lin, Yuxiang Wang, Fangcheng Fu, Bo Du, Quanqing Xu and Jiawei Jiang. How Significant Are the Real Performance Gains? An Unbiased Evaluation Framework for GraphRAG. *arXiv preprint arXiv:2506.06331*, 2025.

- [10] Yilin Xiao, Junnan Dong, Chuang Zhou, Su Dong, Qianwen Zhang, Di Yin, Xing Sun and Xiao Huang. GraphRAG-Bench: Challenging Domain-Specific Reasoning for Evaluating Graph Retrieval-Augmented Generation. *arXiv preprint arXiv:2506.02404*, 2025.
- [11] Pere-Lluís Huguet Cabot and Roberto Navigli. REBEL: Relation Extraction By End-to-End Language Generation. *Findings of the Association for Computational Linguistics: EMNLP*, 2021.
- [12] Darren Edge, Haishi Li, Adrian Doyle, et al. From Local to Global: A GraphRAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- [13] Shahul Es, Jithin James, Luis Espinosa Anke, Steven Schockaert. RAGAS: Automated Evaluation of Retrieval Augmented Generation. *arXiv preprint arXiv:2309.15217*, 2024.
- [14] Xanh Ho, Anh-Khoa Nguyen, Saku Sugawara and Akiko Aizawa. Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. *Proceedings of COLING*, 2020.
- [15] Jim Webber, Emil Eifrem and Ian Robinson. *Graph Databases*. O'Reilly Media, Second Edition, 2015.
- [16] Matthew Honnibal, Ines Montani, Sofie Van Landeghem and Adriane Boyd. spaCy: Industrial-Strength Natural Language Processing in Python. Zenodo, 2020.
- [17] Julian Kahl, Nandan Thakur and Jimmy Lin. BM25S: Orders of Magnitude Faster Lexical Search via Eager Sparse Scoring. *arXiv preprint arXiv:2407.03618*, 2024.
- [18] Ollama. *Ollama: Run Large Language Models Locally*. Disponible en: <https://ollama.com/>, 2024.
- [19] Thomas Wolf, Lysandre Debut, Victor Sanh, et al. Transformers: State-of-the-Art Natural Language Processing. *Proceedings of EMNLP: System Demonstrations*, 38–45, 2020.
- [20] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP-IJCNLP*, 3982–3992, 2019.
- [21] Shitao Xiao, Zheng Liu, Peitian Zhang and Niklas Muennighoff. C-Pack: Packaged Resources To Advance General Chinese Embedding. *arXiv preprint arXiv:2309.07597*, 2023.
- [22] Groq. *Groq: Fast AI Inference*. Disponible en: <https://groq.com/>, 2025.